



College of Computer and Information Sciences

Department of Computer Engineering

Directional Antenna Controller for UAV Data Link

CEN 493- Graduation project

Team Members

Ayman Saleh Taman - 442102757

Abdullah Hamad Alghuzza – 443101414

Mohammed Saleh Althagafi – 443102227

Abdulrahman Amin Mansour - 443106551

2nd Semester 2026

Advisors:

Dr. Abdulmohsen Almutairi & Dr. Salman Alqahtani

Table of Contents

Abstract	4
Chapter 1 The Problem Statement	5
Section 1.1 Need Statement	5
Section 1.2 Objective Statement	5
Chapter 2 Research Survey	6
Section 2.1 Concept of Directional UAV–GCS Communication	6
Section 2.2 Existing Approaches to UAV Tracking and Directionality	6
Section 2.3 Existing Antenna Technologies	7
Section 2.4 Switched-Beam (Sector Selection) for Low-Cost Directionality	7
Section 2.5 SDRs for Prototyping and Measurement	8
Chapter 3 Requirements Specification	9
Section 3.1 User Needs	9
Section 3.2 Requirement Specification	10
Chapter 4 The Design	12
Section 4.1 Concept Overview	12
1. Microcontroller (MCU)	14
2. RF Switch	14
3. Antennas	14
4. GPS Module	15
5. Digital Compass (Magnetometer)	15
6. Altitude	15
7. Power Supply	15
Section 4.2 Design Specifications	16
Design Level 0:	16
Design Level 1:	17
Design Level 2:	18
Chapter 5 Implementation and Verification	25
Section 5.1 Prototype Assembly and Verification	25
1. Hardware Assembly and Interaction	26
2. Software Modules	26

Section 5.2 Design Verification	28
1. UAV Location Sub-System Verification Tests	28
2. RF Switch Verification (LED Test)	31
Section 5.3 Testing Full system with GNU	33
1. Sub-System Verification Tests	34
References	37
Appendix A	40
Section A.1 Work Breakdown Structure	40
Section A.2 Gantt Chart	42
Section A.3 Cost Estimation	42
Appendix B	43
Appendix C	44
Appendix D	47

Abstract

This project presents the design and development of a Directional Antenna Controller intended to enhance the reliability of UAV communication links. Traditional UAV systems commonly rely on omnidirectional or mechanically steered antennas, which introduce limitations such as signal loss, high power consumption, and mechanical wear. To address these challenges, this project proposes an electronically steered antenna system that utilizes a four-sector smart antenna array controlled by microcontroller and RF switch.

The system determines the optimal transmission direction by processing real-time GPS, compass, and altitude data to calculate the geometric bearing between the UAV and the Ground Control Station (GCS). Based on this bearing, the controller dynamically selects the antenna sector that offers the strongest link. The design significantly reduces aerodynamic drag, minimizes power loss in the RF chain, and improves communication stability. This report details the underlying theory, system architecture, hardware selection, and implementation plan. The proposed solution enables a low-cost, lightweight, and maintenance-free alternative to mechanical tracking systems, providing an effective method for improving UAV data-link performance.

Chapter 1 The Problem Statement

The increase of unmanned aerial vehicle (UAV) usage in civilian and industrial applications has created a critical need for a stable and reliable wireless communication link between the UAV and the Ground Control Station (GCS). Conventional UAV links that rely on fixed or omnidirectional antennas often suffer from reduced link quality as distance increases or as the UAV changes orientation. This project proposes a low-cost directional antenna controller that improves the UAV–GCS data link using switched-beam sector selection.

Section 1.1 Need Statement

Over the past few decades, unmanned aerial vehicles (UAVs) have gained popularity in both civilian and military applications, such as surveillance, precision agriculture, disaster management, and mapping. However, establishing a stable and reliable wireless communication channel between the UAV and the ground control station remains challenging, especially in open areas or where there is low traffic. It has been shown that signal distortion or loss can lead to mission failure rates ranging from 10 to 25%, especially when using traditional antennas without sector selection [1].

While commercial solutions such as auto-tracking antennas exist, they rely on mechanically steered systems that physically rotate to follow the UAV, making them costly and prone to failure under harsh environmental conditions. Experiments such as the UAV Beam system have shown that using directional transmission/beam steering approaches significantly enhances performance, achieving a 123.9% throughput increase compared to conventional methods [2]. Therefore, there is a need to develop a low-cost smart antenna system that uses sector selection to ensure a stable and reliable communication channel.

Section 1.2 Objective Statement

This project aims to design a low-cost, lightweight antenna controller that dynamically optimizes the UAV–GCS wireless data link. By processing the vehicle's real-time orientation, the system electronically steers the transmission signal to maintain a direct line to the Ground Control Station. The result is a stable, continuous communication link designed specifically for payload-constrained aerial platforms.

Chapter 2 Research Survey

Modern UAV missions depend on a stable air-to-ground wireless link for telemetry, command/control, and payload data. In practice, link quality varies with range and with UAV orientation because antenna patterns and polarization alignment affect received power. Omnidirectional antennas simplify integration but provide limited link margin, while directional antennas can improve range and robustness if their main lobe remains aligned with the ground control station (GCS). UAV tracking antenna literature emphasizes that tracking solutions must cope with UAV mobility and changing look angles, and it highlights the main architectures used in practice and their trade-offs [3]. This motivates lightweight approaches that improve directionality without adding heavy mechanical systems or high-complexity beamforming hardware.

2.1 Concept of Directional UAV–GCS Communication

Directional communication improves link budget by concentrating radiated energy toward the receiver instead of spreading it in all directions. This is especially valuable for air-to-ground links where path loss increases with distance and where interference may exist in crowded unlicensed bands. However, the gain advantage of a directional antenna is only achieved when the antenna is oriented correctly, which becomes challenging for a UAV that continuously changes heading during flight. Tracking antenna surveys discuss that maintaining correct pointing is a core challenge and show that practical UAV tracking systems typically rely on either mechanical steering or electronic methods to keep a higher-gain beam aligned with the target direction [3].

2.2 Existing Approaches to UAV Tracking and Directionality

Most basic UAV links use omnidirectional antennas because they are small, low cost, and tolerant to rotation, but they typically sacrifice range and robustness. When higher performance is needed, many commercial-style solutions use mechanically steered tracking antennas that rotate in azimuth/elevation to follow the UAV. While effective, this approach adds moving parts and increases cost, maintenance needs, and susceptibility to outdoor conditions such as dust, vibration, and mechanical wear, which is repeatedly highlighted in tracking antenna reviews [3]. Electronic alternatives include phased arrays and beamforming methods that can steer beams without moving parts, but these approaches can increase RF complexity and calibration requirements. Experimental research such as UABeam demonstrates that beamforming with feedback can yield strong performance gains in realistic air-to-ground channels, reporting large throughput improvement when using explicit CSI feedback compared to implicit methods [2]. Other research explores integration of antenna arrays into UAV structures to improve gain while maintaining aerodynamic form factor; for example, a planar printed array embedded in a UAV wing structure was presented for link enhancement, supporting the feasibility of compact high-gain solutions for UAV platforms [4]. Established research shows focus on embedding planar antenna arrays into UAV structures to combine aerodynamic efficiency with high-gain directional communication [4, 5]. The reference project by [4] demonstrated that integrating a 12-element 2.45 GHz array within the UAV wing can achieve up to 21.4 dBi gain, validating the potential of structural integration for UAV data links [4].

2.3 Existing Antenna Technologies

Directional antennas improve link robustness by concentrating radiated energy toward the intended direction. This focused approach significantly increases received signal strength at the target receiver. It also reduces susceptibility to unwanted interference when compared to standard omnidirectional antennas. In UAV communication links, two commonly used directional antenna families are Yagi-Uda antennas and log-periodic dipole array (LPDA) antennas. Each of these options offers different tradeoffs in terms of gain, bandwidth, and integration practicality.

A Yagi-Uda antenna is a directional array built around a single driven element with parasitic elements (typically a reflector and multiple directors) to produce strong forward gain and high directivity. Yagi designs are often favored for fixed-band links because they can provide high gain with a relatively simple structure, but they are generally narrower in bandwidth than wideband array types and can be more sensitive to frequency offset and tuning. These characteristics make Yagi-Uda antennas a common choice when maximum forward gain at a specific operating frequency is the priority [27].

A log-periodic dipole array is designed to maintain usable impedance and radiation characteristics over a wide frequency range by using a self-similar (log-periodic) arrangement of dipole elements. LPDAs provide directional behavior with broadband operation, making them suitable when wide bandwidth is required (e.g., for multi-band systems or flexible testing [28]). In this project, the selected log-periodic antenna option (AN220201-03C) is specified as a wideband directional antenna with a 50-ohm interface and SMA termination, and its mechanical dimensions and mounting requirements are documented in the component datasheet [17].

2.4 Switched-Beam (Sector Selection) for Low-Cost Directionality

A practical middle ground between omnidirectional antennas and full beamforming is a switched-beam or sector-selection approach. Instead of continuously steering a beam, multiple fixed directional antennas (or predefined beams) are arranged to cover the full azimuth plane, and the system selects the sector that best aligns with the desired direction. This reduces mechanical complexity and avoids continuous phase-controlled beam steering while still providing directional gain compared to a single omnidirectional radiator [5]. For UAV data links specifically, an automatic sector-switching architecture has been described that uses a controller and a multiport RF switch to select among antenna sectors based on periodic updates, enabling directional operation without mechanical tracking [6]. The main limitations of sector selection are the finite angular resolution (limited by sector width) and potential rapid switching near boundaries unless the control logic includes stabilization (such as hysteresis or filtering), which must be considered during implementation and testing [6].

2.5 SDRs for Prototyping and Measurement

Software-defined radios (SDRs) are widely used to prototype and validate wireless systems because many radio functions can be implemented in software, enabling rapid experimentation and measurement without redesigning hardware. Foundational SDR literature describes how radio platforms can be reconfigured through software, supporting flexible testing of modulation, filtering, and measurement tasks [7]. In UAV link research, SDR-based platforms are also used to perform realistic air-to-ground measurements and evaluate performance under mobility, as shown in UABeam’s testbed and field evaluation methodology [3]. For this project, SDR-based testing provides a repeatable method to quantify the effect of sector switching on received power and link quality and to compare sector selection against a fixed-antenna baseline under controlled conditions.

Chapter 3 Requirements Specification

Section 3.1 User Needs

1. The system must be able to adapt its radiation direction toward a ground control station (GCS).
2. The system must improve communication link reliability and range.
3. The system should automatically maintain the best antenna direction toward the GCS during flight.
4. The system must be small, lightweight, and low-profile to support UAV mounting with minimal aerodynamic impact.
5. The system must have low power consumption and support common UAV power sources (battery/regulator compatibility).
6. The system should provide full coverage to support UAV motion in any heading.
7. The system should operate reliably in outdoor flight conditions (temperature/vibration within UAV environment).

Section 3.2 Requirement Specification

ID	Engineering Requirement	Justification	Linked user Need(s)
1	The system shall compute bearing and command antenna-sector switching with end-to-end latency ≤ 500 ms from new fused sensor data availability (GNSS + heading).	Keeping the sensing-to-switching loop within 500 ms allows the controller to track GCS bearing changes during flight and automatically select the most suitable antenna sector, improving link robustness and range, while remaining achievable under typical GNSS update rates and stability filtering.	1, 2, 3
2	The system shall operate from -40°C to $+85^{\circ}\text{C}$ with no loss of intended function, including correct sector selection and switching.	According to MIL-STD-810, embedded systems in military UAVs must operate between -40°C to $+85^{\circ}\text{C}$.	7
3	The complete antenna module shall have a maximum vertical height of 20 mm and a total weight not exceeding 400 grams.	20 mm ensures the device fit with UAV, 400 grams ensures negligible impact on UAV flight time and battery life.	4, 5
4	The selected antenna shall provide a minimum peak Gain of 5 dBi in the direction of the GCS.	A minimum 5 dBi peak gain improves the link budget toward the GCS, helping extend range and increase communication reliability.	2
5	The system shall provide a front-to-back ratio of at least 12 dB.	A front-to-back ratio ≥ 12 dB reduces radiation in the opposite (unwanted) direction, improving link focus toward the GCS and helping reduce interference.	2
6	The system shall maintain an antenna pointing accuracy of ± 45 degrees	Maintains link quality and improves reliability.	1, 3
7	The system shall consume no more than 2.5 Watts of power during active transmission and processing.	The limit ensures the module remains a low-impact payload in terms of energy consumption, helping preserve UAV endurance while running continuously during a mission.	2, 5, 7

Marketing Requirements:

1. The system must be able to adapt its radiation direction toward a ground control station (GCS).
2. The system must improve communication link reliability and range.
3. The system should automatically maintain the best antenna direction toward the GCS during flight.
4. The system must be small, lightweight, and low-profile to support UAV mounting with minimal aerodynamic impact.
5. The system must have low power consumption and support common UAV power sources (battery/regulator compatibility).
6. The system should provide full coverage to support UAV motion in any heading.
7. The system should operate reliably in outdoor flight conditions (temperature/vibration within UAV environment).

Chapter 4 The Design

This chapter details the design and implementation of the airborne system installed on the Unmanned Aerial Vehicle (UAV) for reconnaissance and data link management. The core concept relies on a custom antenna array managed by a directional antenna controller. The primary function of this controller is to compute the geometric bearing to the Ground Control Station (GCS) using GPS and compass data and to steer transmission by selecting the appropriate antenna sector. The following sections outline the concept overview, component selection, and detailed design specifications.

Section 4.1 Concept Overview

The airborne system mounted on the UAV consists of several integrated hardware components designed to enable directional wireless communication. The core of the system is a Microcontroller Unit (MCU) that is responsible for processing sensor data and controlling the antenna selection logic. An RF switch is used to route the transmitted RF signal to one of multiple directional antennas installed on the UAV structure. The MCU receives real-time navigation data from a GPS module, a digital compass (magnetometer), and an altitude sensor, which together provide accurate position and orientation information. The directional antennas are strategically placed on different sides of the UAV to ensure full 360-degree coverage while maintaining low weight and minimal aerodynamic impact.

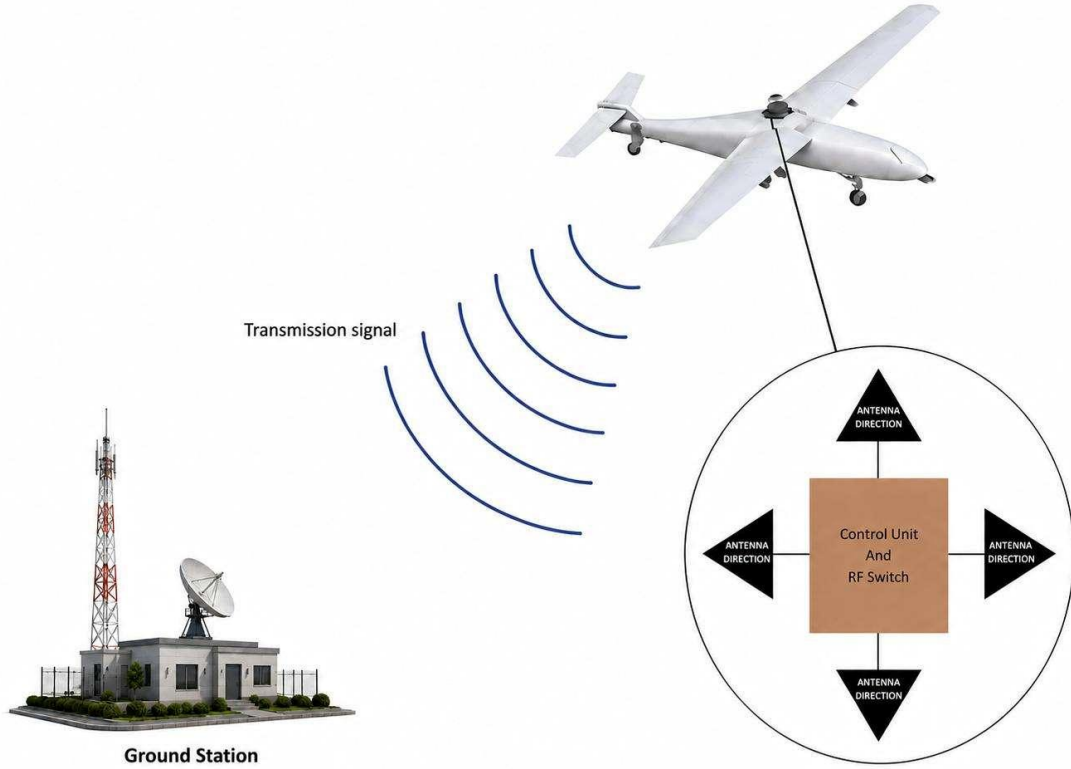


Figure 1: Concept overview

The illustration presents a conceptual overview of the directional communication link between the UAV and the Ground Control Station (GCS). The UAV is shown equipped with a directional antenna controller that transmits a focused wireless signal toward the GCS instead of radiating energy uniformly in all directions. The signal propagation is visualized using directional wave patterns, emphasizing the concept of beam concentration toward the intended receiver. On the ground side, the GCS is depicted with a fixed receiving station that represents the endpoint of the air-to-ground data link. This figure visually demonstrates how directional transmission improves link efficiency and communication reliability compared to omnidirectional approaches.

The system operates by continuously coordinating between sensing, processing, and RF switching components to maintain an optimal communication link. First, the onboard navigation sensors provide the MCU with updated position, heading, and altitude data during flight. The MCU then computes the relative bearing between the UAV and the pre-defined location of the GCS. Based on this calculated bearing, the MCU determines which directional antenna offers the best transmission path. Finally, the MCU commands the RF switch to activate the selected antenna, routing the outgoing RF signal accordingly and ensuring a stable and efficient data link throughout UAV movement.

Table 1: Concept Table

MCU	RF switch	Antennas	GPS	Altitude	Compass
ESP32-s3	EV1HMC241AQS16	AN220201-03C	NEO-M8N	BMP390	MMC5983MA
STM32-S3	SKY13370-370LF	Yagi-Uda Antenna	RUSHFPV GNSS MINI M10	LPS22HB	BMM150
Arduino Mega 2560	QPC8019Q	Taoglas WLP.2450.2 5.4.A.02	Beitian BN-880	MS5611	HMC5883L

4.1.1 Microcontroller (MCU)

The options considered are ESP32-S3[22], STM32[9], and Arduino Mega 2560[8]. Based on the AHP analysis, the microcontroller selection was evaluated using four weighted criteria: real-time control (0.45), size (0.25), performance (0.20), and cost (0.10). Real-time control was prioritized to ensure reliable sensor acquisition (GNSS/IMU/barometer) and deterministic RF-switch actuation with low latency. Size was considered to support compact UAV integration and reduce wiring/auxiliary hardware. Performance captured the processing capability required for bearing computation, filtering, and the necessary peripheral throughput (UART/I²C/SPI/GPIO). Cost was included as a secondary factor due to the higher priority of timing and integration constraints. Using these criteria and weights, the ESP32-S3 achieved the highest overall score and was selected as the controller for the directional antenna switching system.

4.1.2 RF Switch

The options considered are Ev1hmc241AQS16 [16], SKY13370-370LF[25], and QPC8019Q[26]. The selection criteria focused on insertion loss and isolation to preserve link budget, as well as port configuration suitability for connecting four antennas (SP4T or equivalent). Additional considerations included control interface requirements (logic/biasing needs), power handling, PCB/layout complexity, and overall cost. Based on these criteria, EV1HMC241AQS16 was selected as the best choice due to its low-loss switching performance and strong isolation characteristics, while still meeting the required control and configuration needs.

4.1.3 Antennas

The options considered are Log-Periodic (AN220201)[17], Yagi-Uda[23], and Ceramic Patch[24]. The selection criteria prioritized gain and beamwidth/pattern suitability to support sector-based coverage, while ensuring adequate bandwidth at the operating frequency. The selection also accounted for size/weight and aerodynamic impact, mounting practicality and robustness, and pattern consistency when installed on the UAV (including coax routing and placement effects). Based on these criteria, the Log-Periodic antenna (AN220201) was chosen as it provides a strong balance of gain and wideband performance with a flat, aerodynamic form factor suitable for UAV integration.

4.1.4 GPS Module

The options considered are u-blox NEO-M8N[21], RUSHFPV GNSS M10[10], and Beitian BN-880[11]. The selection criteria emphasized position accuracy and stability (to ensure reliable bearing computation), along with update rate and UART interface compatibility for integration with the controller. Practical integration factors such as size/weight for UAV mounting, integration/support maturity (documentation and predictable behavior), and cost/availability were also included. Based on these criteria, the u-blox NEO-M8N was selected as it provides stable positioning performance with straightforward integration and well-established documentation.

4.1.5 Digital Compass (Magnetometer)

The options considered are MMC5983MA[19], BMM150[12], and HMC5883L[13]. The selection criteria prioritized resolution and noise performance to improve heading stability, in addition to measurement stability/repeatability under UAV operating conditions. The selection also considered update rate, susceptibility to magnetic interference (and resulting calibration complexity), I²C interface compatibility, and availability/lifecycle status to avoid discontinued components. Based on these criteria, MMC5983MA was selected due to its superior resolution and stability characteristics, with the understanding that careful placement and calibration are required in the UAV environment.

4.1.6 Altitude

The options considered are BMP390[20], LPS22HB[14], and MS5611[15]. The selection criteria focused on pressure/altitude precision and drift, as well as noise performance and power consumption for continuous operation. Additional considerations included update rate, package size and integration effort, and sensitivity to airflow/prop wash (which drives mounting and shielding requirements). Cost and availability were also considered to ensure feasibility. Based on these criteria, BMP390 was selected due to its strong precision and low drift performance with low power consumption, while remaining practical to integrate within the overall system.

4.1.7 Power Supply

The MB102 Power Supply Module was selected as an alternative power source during the prototyping and testing phase due to the current unavailability of the UAV. The selection criteria focused exclusively on voltage stability and reliability. The MB102 utilizes integrated AMS1117 LDO regulators to provide clean and stable 5V and 3.3V rails, essential for powering the ESP32-S3, the RF Switch, and the navigation sensor array safely and efficiently during laboratory experiments.

Section 4.2 Design Specifications

This section explains the system's architecture and presents a detailed design description of the proposed solution. It begins by introducing the overall system structure and how the major modules interact to achieve directional antenna switching. The functional specifications are then outlined for each module in a structured manner. The description starts with the high-level Level 0 design and gradually refines the system into lower-level functional blocks. Finally, it progresses to the detailed design of the main components, clarifying the inputs, outputs, and role of each module within the complete system.

Design Level 0:

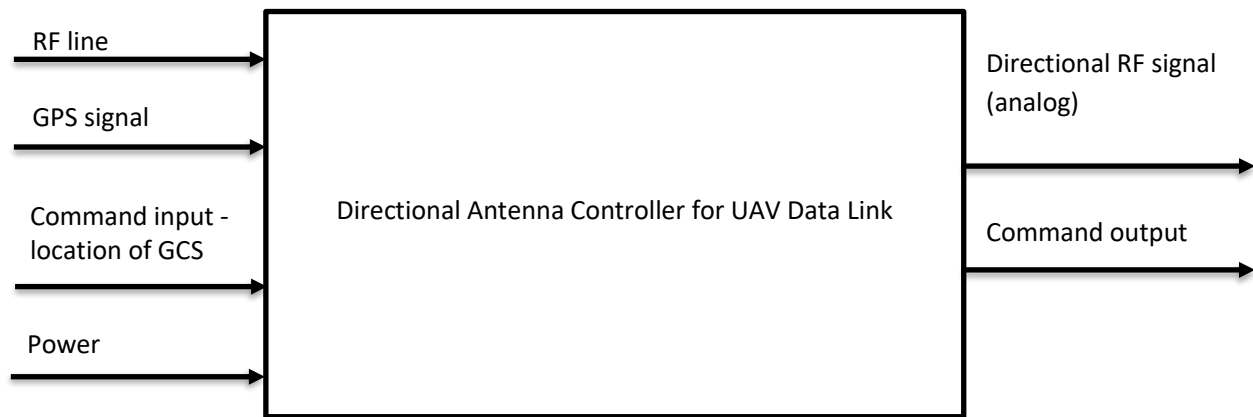


Figure 2: Design level 0

Table 2: Directional Antenna Controller for UAV Data Link

Module	Directional Antenna Controller for UAV Data Link
Inputs	- RF Line: The data signal (payload) to be transmitted. - GPS Signal: The UAV's current location data. - Command Input - Location of GCS: The Ground Control Station's location data. - Power : Input power supplied to the system.
Outputs	- Directional RF Signal (analog): The final transmission signal directed to the GCS. - Command Output: Acknowledgment (ACK) and system status signals sent back to the GCS or user interface (e.g., over Bluetooth).
Functionality	The module uses UAV and GCS coordinates to calculate the optimal direction, selecting the best of four antennas via an RF Switch for transmission.

Design Level-1:

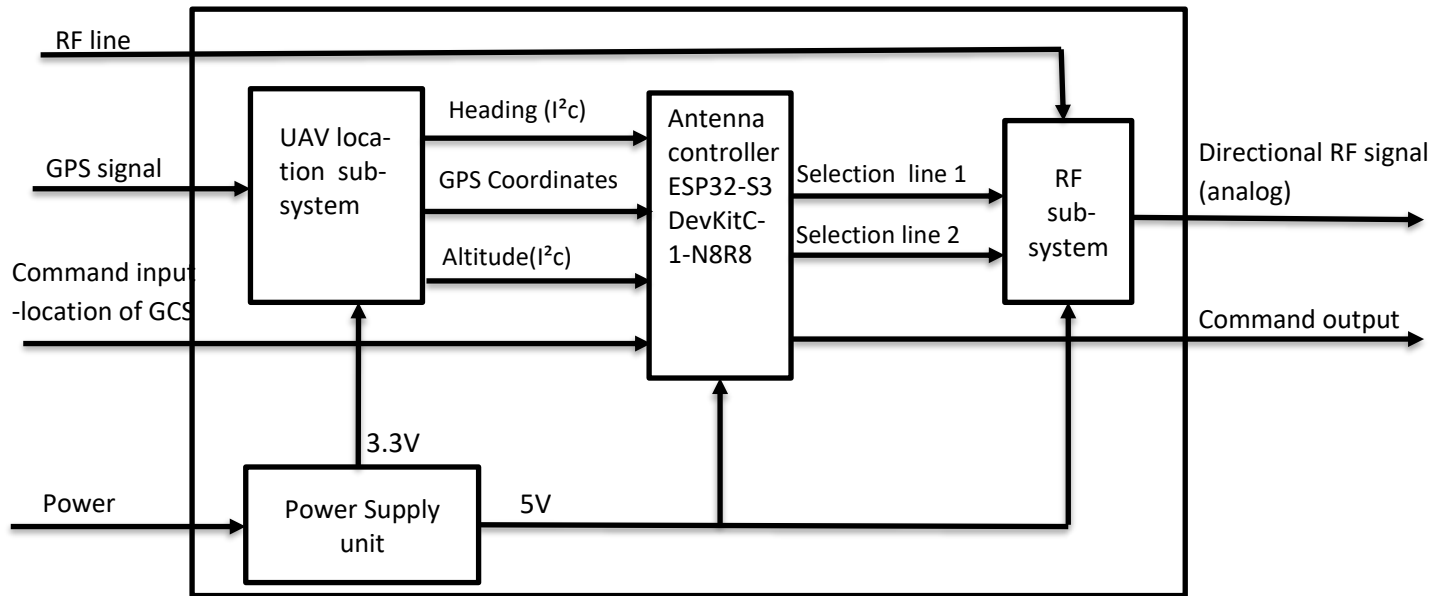


Figure 3: Design level 1

Table 3: RF Sub-System

Module	RF Sub-System
Inputs	- RF Line: The data signal (payload) to be transmitted. - Selections Line 1 & 2: Two control signals from the MCU. - Power: 5V from Power supply unit
Outputs	Directional RF Signal (analog): Signal sent to GCS.
Functionality	The controller will send a control signal to the RF switch to activate a certain sector of the antenna array, which will then send the signal to the GCS.

Table 4: UAV Location Sub-System

Module	UAV Location Sub-System
Inputs	- Power: 3.3V from Power Supply Unit. - GPS Signal: The UAV's current location data.
Outputs	- Heading (I²C). - GPS Coordinates (UART data payload). - Altitude (I²C).
Functionality	The module will find the heading, location, and altitude of the UAV which will be forwarded to the MCU.

Table 5: Antenna Controller ESP32-S3

Module	Antenna Controller ESP32-S3
Inputs	- Heading (I²C) - GPS Coordinates (UART data payload) - Altitude (I²C) - Power: 5V from Power supply unit
Outputs	Selections line 1 & 2: Control lines to the RF switch (antenna sector selection). Selections Output: An Acknowledgment (ACK) message confirming that the new GCS location data has been successfully received and updated.
Functionality	Processes UAV and GCS coordinates to compute the relative bearing, selects the optimal antenna sector, and drives the RF switch.

Table 6: Power Supply Unit

Module	Power Supply Unit
Inputs	Power: To charge the power bank
Outputs	- 3.3V For UAV Location Sub-System - 5V For Antenna Controller ESP32-S3 and RF Sub-System
Functionality	Acts as the primary power distribution center for the system. It supplies 5V directly to the main controller and RF switch and utilizes an AMS1117 LDO regulator to step down the voltage, providing a clean and stable 3.3V to the sensitive navigation sensors.

Design Level 2:

1) UAV Location Sub-System

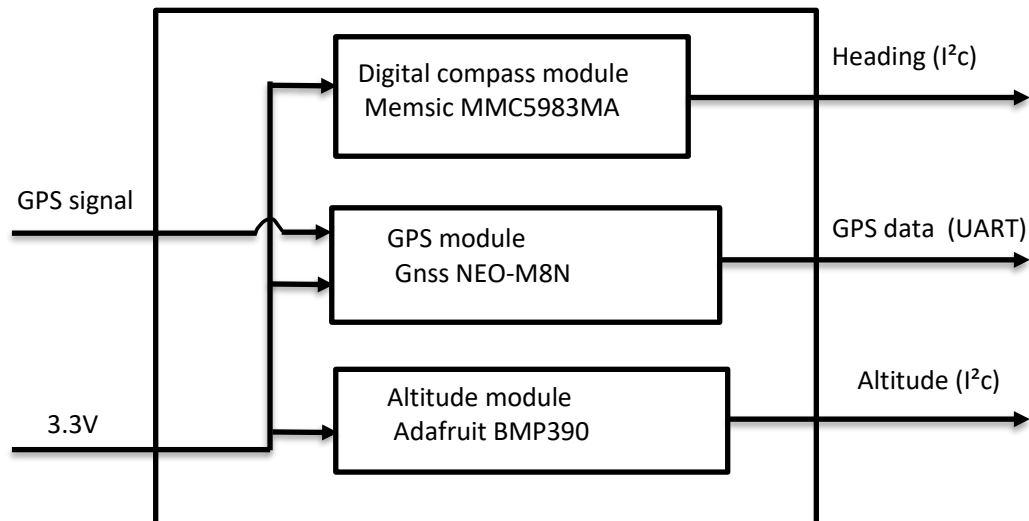


Figure 4: Design level 2 UAV LOCATION Sub-System

Table 7: Altitude Module Adafruit BMP390

Module	Altitude Module Adafruit BMP390
Inputs	Power: 3.3V
Outputs	Altitude (I ² C)
Functionality	Uses barometric pressure and provides temperature measurements to derive high-resolution altitude estimates.

Table 8: Digital Compass Module Memsic MMC5983MA

Module	Digital Compass Module Memsic MMC5983MA
Inputs	Power: 3.3V
Outputs	Heading (I ² C)
Functionality	Generates precise 3-axis magnetic field data used to determine the UAV's heading. When fused with GPS and inertial sensors, it enhances orientation accuracy and directional stability during flight.

Table 9: GPS Module Gnss NEO-M8N

Module	GPS Module Gnss NEO-M8N
Inputs	- Power: 3.3V - GPS Signal
Outputs	GPS Coordinates (UART Data Payload)
Functionality	Delivers real-time GPS position, velocity, and timing data essential for navigation. It enables accurate trajectory tracking and supports antenna-pointing algorithms by providing continuous geographic updates.

2) RF Sub-System

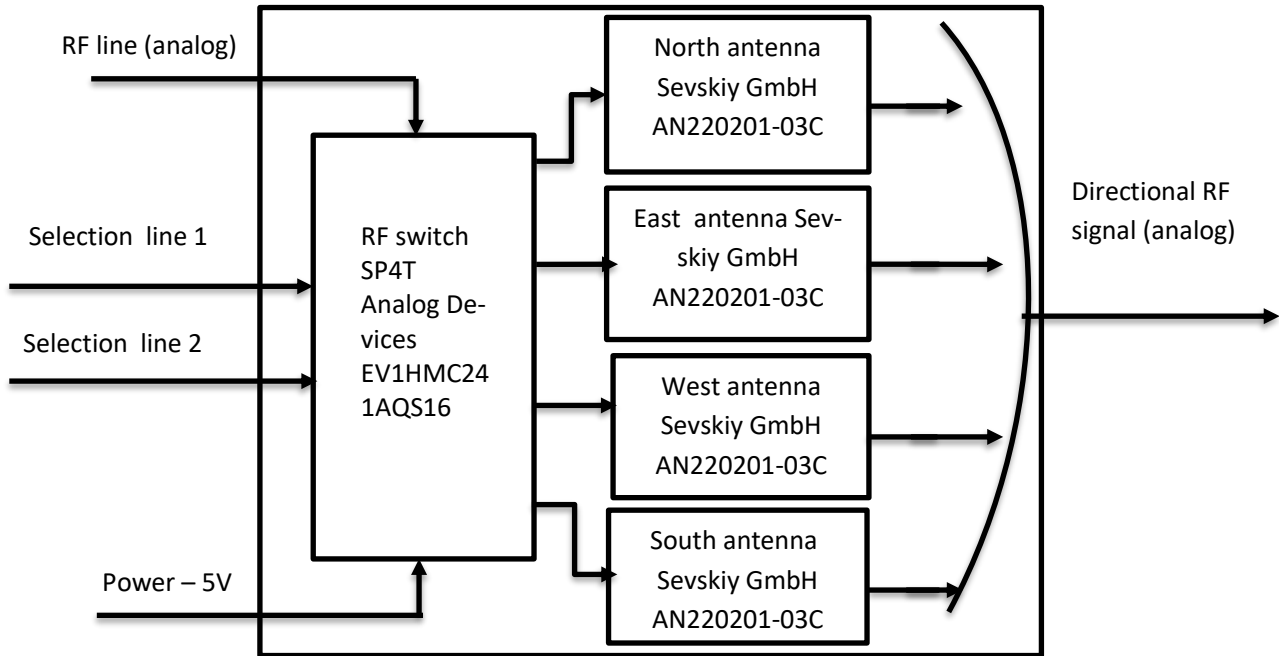


Figure 5: Design level 2 RF Sub-System

Table 10: RF Switch SP4T Analog Devices EV1HMC241AQS16

Module	RF Switch SP4T Analog Devices EV1HMC241AQS16
Inputs	- Power: 5V from the Power Supply Unit. - RF Line (analog): Signal coming from the radio. - Selection Line 1 & 2: Two control signals from the MCU.
Outputs	Four ports connected to each antenna sector.
Functionality	The controller will send a control signal to the RF switch to activate a certain sector of the antenna array and will send the signal to said antenna sector.

Table 11: Antenna Sevskiy GmbH AN220201-03C

Module	Antenna Sevskiy GmbH AN220201-03C
Inputs	Signal from RF switch headed to the selected antenna.
Outputs	Directional RF Signal (analog): Signal from a specific antenna sector.
Functionality	A certain antenna will be activated based on the choice from the RF switch and will then send the signal to the GCS.

3) Power Supply Unit

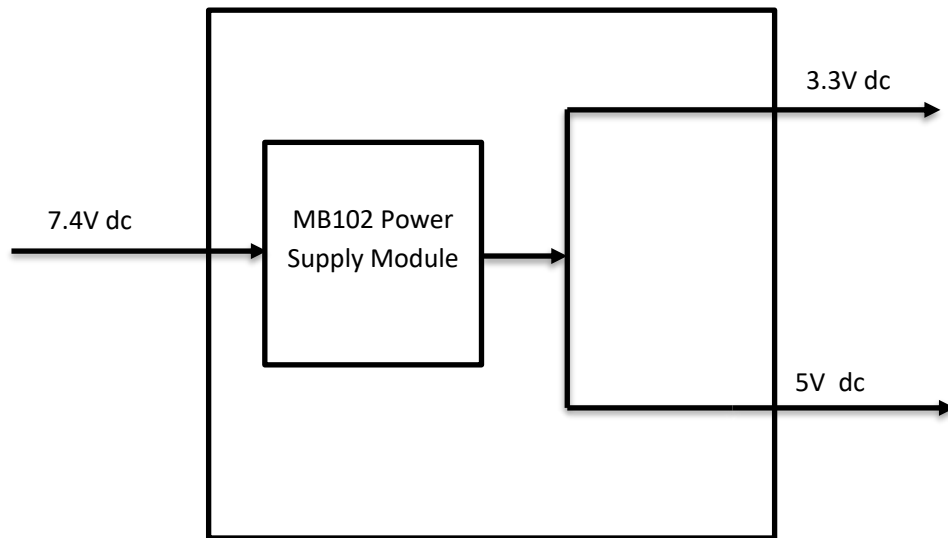


Figure 6: Design level 2 Power Supply Unit

Table 12: Power Supply Unit

Module	MB102 Power Supply Module
Inputs	- 2× 18650 Li-ion batteries in series, ~7.4V DC
Outputs	- 5V rail - 3.3V rail
Functionality	- Delivers stable 5V and 3.3V power rails for breadboard-based circuits through onboard AMS1117 voltage regulators, allows voltage selection for each rail using jumpers, and is suitable for powering microcontrollers such as ESP along with connected sensors.

4) Antenna Controller

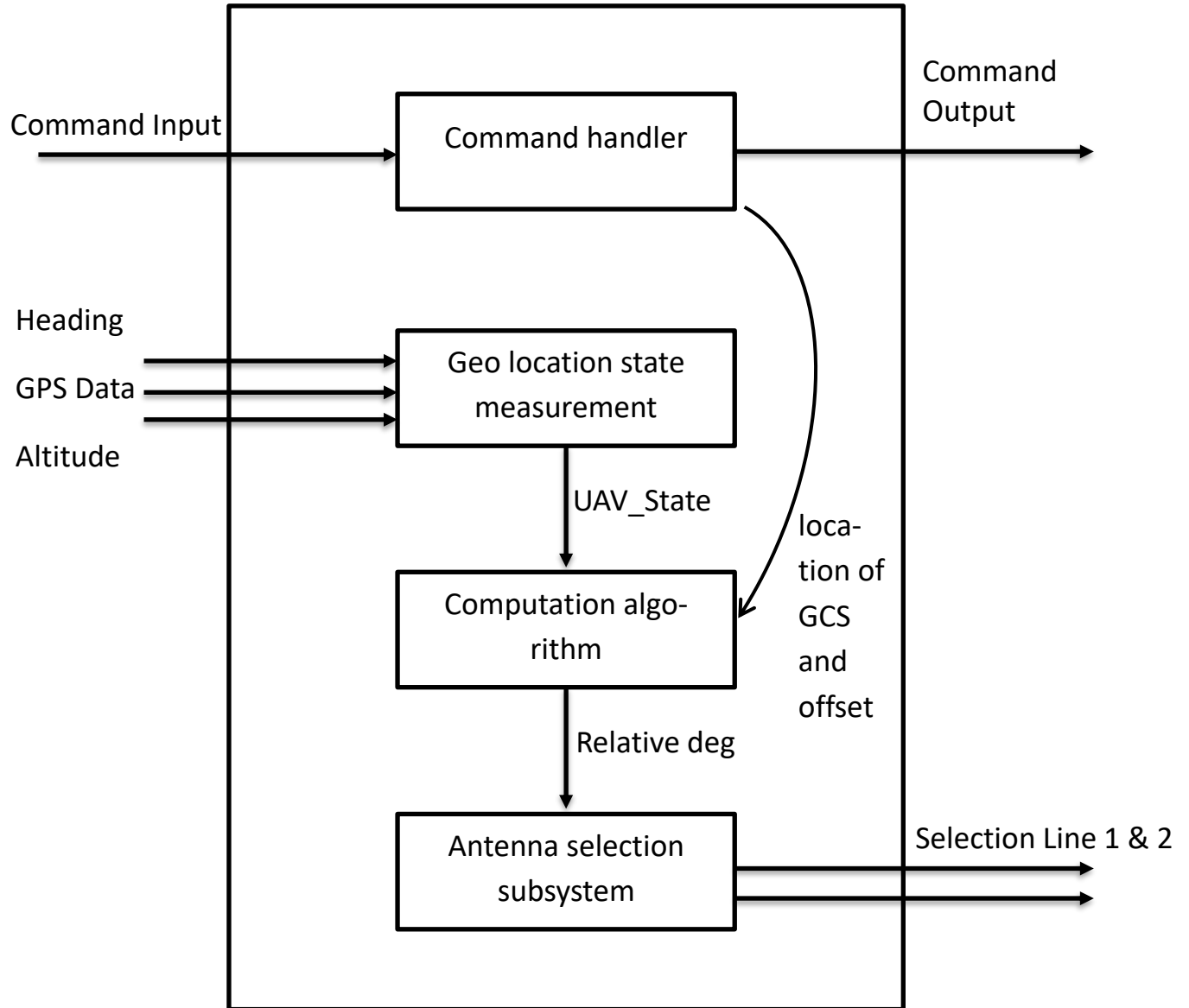


Figure 7: Design level 2 antenna Controller

Table 13: Command Handler

Module	Command Handler
Inputs	Command Input: Bluetooth link, messages containing GCS location update (lat, lon, alt).
Outputs	Command output (ACK/status back over Bluetooth), location of GCS (stored + forwarded to Computation Algorithm) and offset (Compass calibration).
Functionality	Accepts a Bluetooth connection from a phone/GCS app, parses commands, validates lat/lon ranges, updates the saved GCS coordinates.

Table 14: Geo location state measurement

Module	Geo location state measurement
Inputs	- GPS data from UAV. - Heading data from the Compass/Magnetometer. - Altitude data from Barometer/Altimeter
Outputs	UAV State: (lat,lon,altitude. Heading).
Functionality	Reads the onboard sensors and conditions the measurements to produce the UAV state used by the calculation module.

Table 15: Computation algorithm

Module	Computation algorithm.
Inputs	- UAV State (lat, lon, altitude, heading). - location of GCS. - offset
Outputs	Relative deg (direction to the GCS relative to the UAV heading).
Functionality	Computes the bearing from the UAV to the GCS using GPS coordinates, then converts it to a relative azimuth angle using the UAV heading for sector selection.

Table 16: Antenna selection subsystem

Module	Antenna selection subsystem.
Inputs	Relative deg.
Outputs	Selection Line 1 & 2: RF switch control line 1 and control line 2.
Functionality	Maps the relative azimuth angle to the correct antenna sector and drives the RF switch control pins to activate the selected antenna.

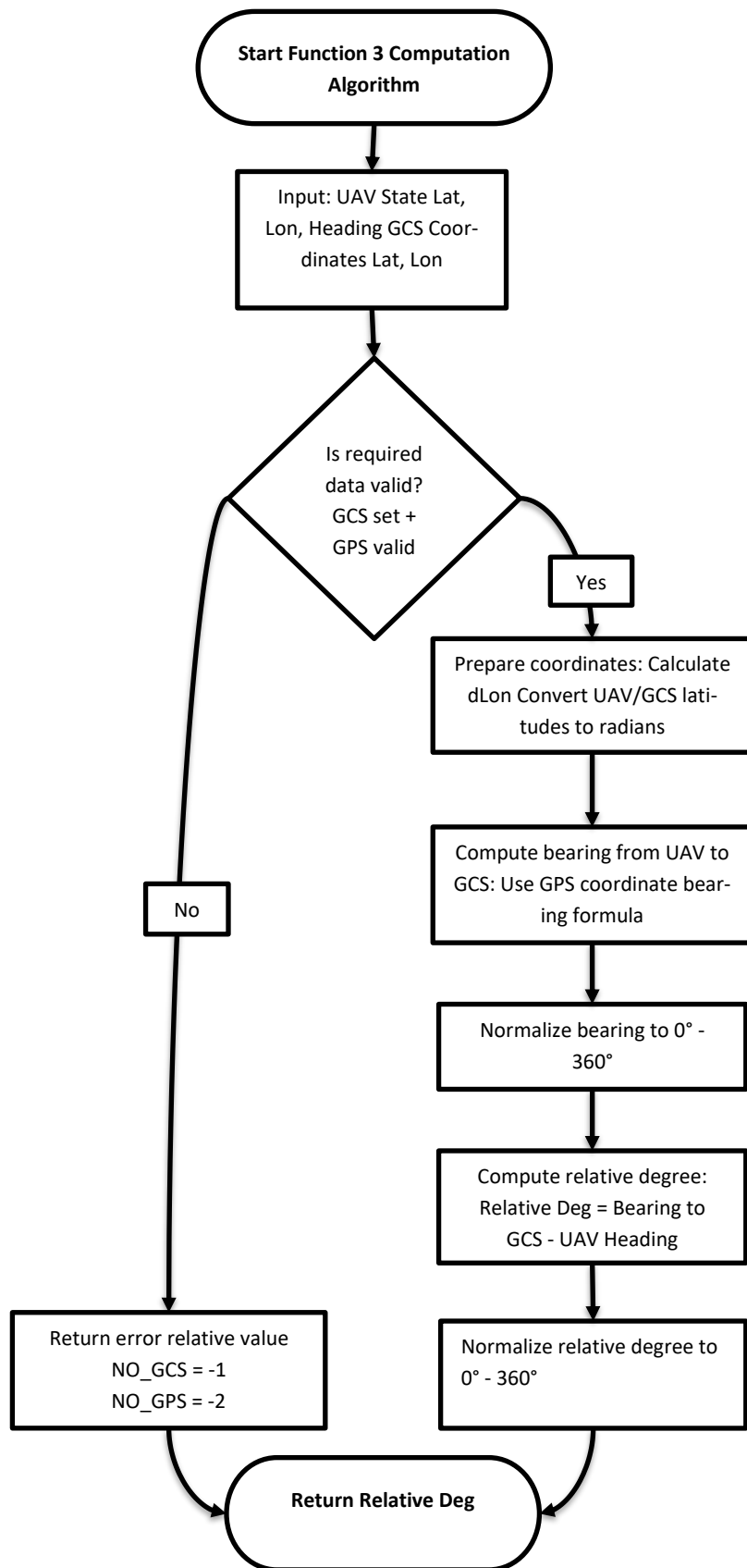


Figure 8: Main Algorithm Flowchart

Chapter 5 Implementation and Verification

This chapter outlines the practical realization and physical assembly of the Directional Antenna Controller for the UAV Data Link. It transitions the system from the conceptual design into a fully functional prototype by detailing the step-by-step hardware integration and software deployment. Furthermore, the chapter documents the rigorous verification procedures—utilizing a Software-Defined Radio (SDR) testbed to systematically validate that the assembled prototype meets all predefined engineering requirements, including sensor accuracy, signal gain, and system latency.

Section 5.1 Prototype Assembly and Verification

The prototype realization was executed through a systematic assembly process, marrying the hardware components with the developed firmware to achieve real-time directional communication. The assembly was conducted in the following sequential stages.

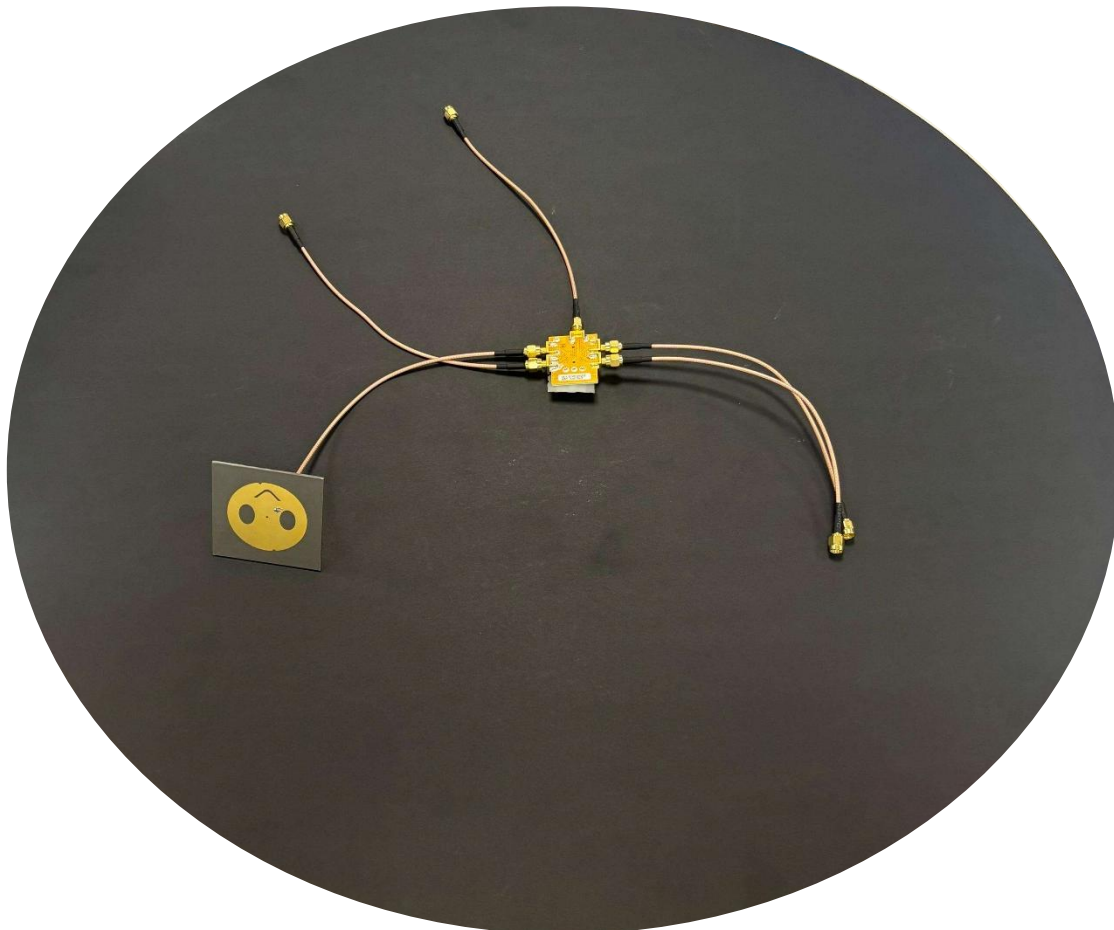


Figure 9: Final Prototype Assembly of the Directional Antenna Controller

5.1.1 Hardware Assembly and Interaction

The hardware prototype was assembled using a breadboard to facilitate the manual wiring of the individual components, with the ESP32-S3 microcontroller acting as the central processing unit. The sensors (GPS, Altitude, and Compass) were manually wired to the MCU's I²C and UART pins to continuously feed spatial data. For the RF routing, the MCU's GPIO pins were connected to the RF switch to act as selection lines. A battery pack utilizing two 18650 Li-ion cells, combined with a voltage regulator module, provided the necessary 5V and 3.3V rails to power the MCU, sensors, and the SDR simultaneously. Finally, the Software-Defined Radio (SDR) was connected via co-axial cables to the RF switch, which in turn routed the signal to the selected directional antenna.

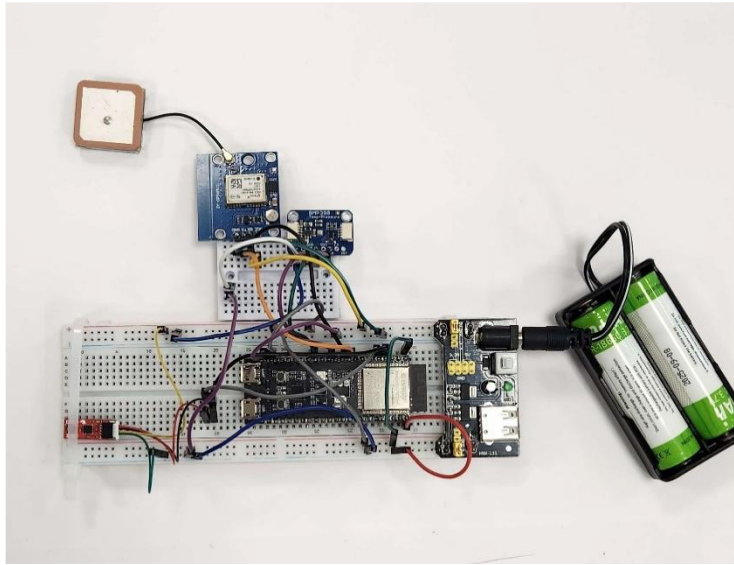


Figure 10: Final Prototype Assembly of the Directional Antenna Controller without RF Switch

5.1.2 Software Modules

This section presents the software implementation used to support the Directional Antenna Controller prototype. The software side is divided into two main parts: the GNU Radio flowgraph used for SDR-based RF signal generation and measurement, and the C++ code uploaded to the ESP32-S3 microcontroller. The GNU Radio implementation supports the RF testing side of the project by allowing the transmitted signal, received signal, and antenna-sector behavior to be observed during switching. The C++ implementation controls the embedded hardware by reading the navigation sensors, processing the UAV location and heading data, and commanding the RF switch through digital selection lines. Together, both software modules allow the prototype to sense the UAV direction, select the correct antenna sector, and verify the RF behavior of the selected path.

Microcontroller Firmware (C++)

The C++ code runs on the ESP32-S3 and operates as the main embedded control program for the prototype. It reads the GPS coordinates from the GPS module through UART and receives heading and altitude data from the compass and altitude sensor through the I²C interface. The code then processes these inputs to calculate the relative direction between the UAV and the Ground Control Station. Based on this calculated direction, the program determines the suitable antenna sector and outputs the corresponding digital logic signals through two GPIO pins connected to the RF switch. These selection lines cause the RF switch to route the RF signal to the selected directional antenna. In the project, the C++ code provides the real-time control link between the UAV Location Subsystem and the RF Subsystem, allowing sensor readings to be converted into actual antenna-sector switching commands.

Section 5.2 Design Verification

This section presents the results of testing the system's prototype to verify the design requirements. Verification includes component-level testing and a primary system acceptance test to demonstrate the directional switching capability.

5.2.1 UAV Location Sub-System Verification Tests

Before conducting the main acceptance test, individual sub-systems were verified to ensure proper operation.

1. Sensor Verification:

- **GPS (NEO-M8N):** Verified by comparing the UART output coordinates against Google Maps. Despite the 20-meter limitation mentioned earlier, the data was successfully parsed by the MCU.

Table 17: Functional Test Case 01 - Verification of GPS Coordinate Acquisition and Update Accuracy

Test Writer: Ayman and Abdullah						
Test case name		GPS Data Reception		Test ID#:		01
Description:		Verify that the GPS module correctly sends UAV coordinates to the ESP32-S3.				
Tester Information						
Name of Tester:		Ayman and Abdullah		Date:		19/4/2026
Hardware Ver:		ESP32-S3 + NEO-M6N		Time:		11:00 pm
Setup:		Connect GPS module via UART and power system with 3.3V rails.				
Steps	actions	Expected Result	Pass	Fail	N/A	Comments
1	Power on system	ESP32 boots successfully	✓			
2	Initialize NEO-M6N	NEO-M6N detected on UART bus	✓			The NEO-M6N was used in place of the NEO-M8N due to hardware availability constraints.
3	Wait for GPS lock	GPS obtains satellite fix	✓			
4	Read UART output	Latitude and longitude displayed correctly	✓			Coordinates verified against Google Maps — values matched actual location
5	Move GPS position	Coordinates update in real time	✓			Walked with device; Google Maps confirmed position updated correctly
Overall test result :			✓			GPS coordinates consistent with Google Maps in all tests

Altitude (BMP390): Verified by moving between different floor elevations and comparing the I²C output to a commercial altimeter watch.

Table 18: Functional Test Case 02 - Verification of Barometric Pressure and Altitude Estimation Stability

Test Writer: Ayman and Abdullah						
Test case name		BMP390 Pressure & Altitude Reading		Test ID#:		02
Description:		Verify that the BMP390 module correctly sends pressure and altitude data to the ESP32-S3.				
Tester Information						
Name of Tester:		Ayman and Abdullah		Date:		19/4/2026
Hardware Ver:		ESP32-S3 + BMP390		Time:		11:00 pm
Setup:		Connect BMP390 module via I2C and power with 3.3V rail.				
Steps	actions	Expected Result	Pass	Fail	N/A	Comments
1	Power on system	ESP32 boots successfully	✓			
2	Initialize BMP390	BMP390 detected on I2C bus	✓			
3	Read sensor output	Pressure and altitude values displayed correctly	✓			Altitude reading compared with watch — values were consistent
4	Change altitude	Altitude value updates in real time	✓			Moved to different floor; watch confirmed altitude change matched sensor output
Overall test result :			✓			BMP390 altitude readings consistent with watch altimeter across all tests

Compass (MMC5983MA): Verified by rotating the prototype and cross-checking the MCU heading output with a smartphone compass.

Table 19: Functional Test Case 03 - Verification of Magnetometer Heading Detection and Orientation Reliability

Test Writer: Ayman and Abdullah						
Test case name		Compass Heading Detection			Test ID#:	03
Description:		Verify magnetometer heading readings are correctly processed by MCU.				
Tester Information						
Name of Tester:		Ayman and Abdullah			Date:	19/4/2026
Hardware Ver:		ESP32-S3 + MMC5983MA			Time:	11:00 pm
Setup:		Connect compass module over I2C.				
Steps	actions	Expected Result	Pass	Fail	N/A	Comments
1	Power on system	ESP32 boots successfully	✓			
2	Initialize MMC5983MA	MMC5983MA detected on I2C bus	✓			
3	Rotate UAV proto-type	Heading changes correctly	✓			Heading values compared with phone compass app — direction matched correctly
4	Observe serial monitor	Values remain stable with low noise	✓			Readings observed on serial monitor showed minimal fluctuation
5	Compare with real direction	Error within acceptable range	✓			Heading cross-checked with both watch compass and phone compass — error was within acceptable range
Overall test result :			✓			MMC5983MA heading readings consistent with watch and phone compass in all tests

5.2.2 RF Switch Verification (LED Test)

To safely verify the RF switch selection logic without risking the SDR components, an LED test was implemented. LEDs were temporarily connected to the active RF paths. When the MCU was commanded to switch sectors (Front, Right, Back, Left) via the orientation algorithm, the corresponding LED illuminated, confirming that the digital GPIO control lines successfully routed the physical path on the EV1HMC241AQS16 switch.

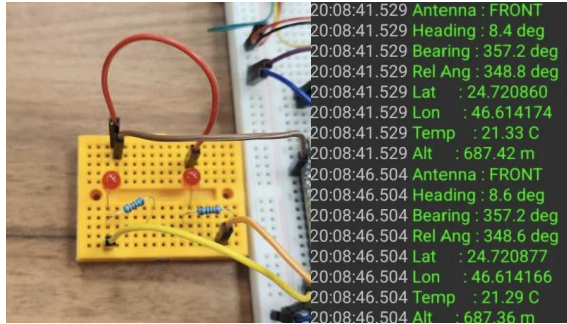


Figure 11: LED Indication and Serial Output Confirming Front-Sector Activation

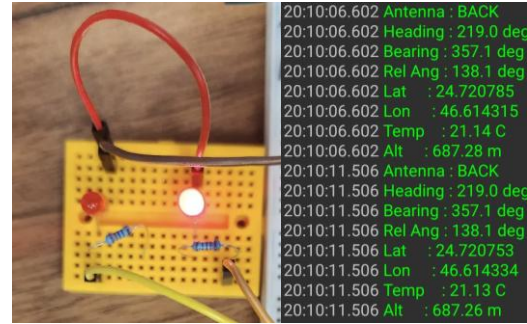


Figure 12: LED Indication and Serial Output Confirming Back-Sector Activation

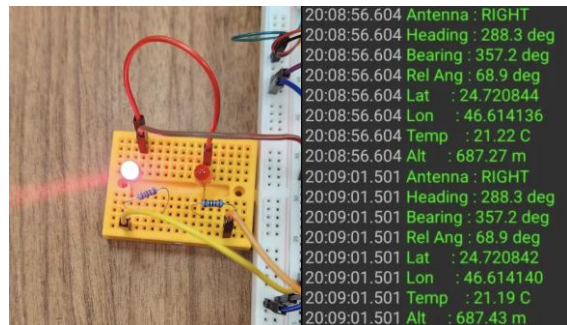


Figure 13: LED Indication and Serial Output Confirming Right-Sector Activation

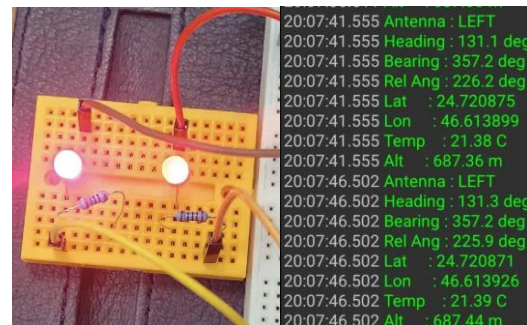


Figure 14: LED Indication and Serial Output Confirming Left-Sector Activation

Table 20 : RF Switch antenna selection truth table

	A	B
Front (Fig 11)	0	0
Back (Fig 12)	0	1
Right (Fig 13)	1	0
Left (Fig 14)	1	1

Table 21: Functional Test Case 04 - Verification of RF Switch Sector Selection Logic Using LEDs

Test Writer: Ayman and Abdullah							
Test case name		Real-Time Antenna Selection Using LEDs			Test ID#:		04
Description:		Verify that the ESP32-S3 correctly updates antenna sector selection in real time while the prototype is continuously rotated.					
Tester Information :							
Name of Tester:		Ayman and Abdullah			Date:		15/5/2026
Hardware Ver:		ESP32-S3 + LED + UAV Location Sub-System			Time:		8:00 pm
Setup:		Verify that the ESP32-S3 correctly updates antenna sector selection in real time while the prototype is continuously rotated.					
Steps	actions	Expected Result	Pass	Fail	N/A	Comments	
1	Power on system	ESP32 boots successfully	✓				
2	Initialize sensors and GPIO outputs	Compass and LEDs detected successfully	✓				
3	Continuously rotate prototype clockwise	Heading values update continuously	✓			Serial monitor reflected smooth heading updates	
4	Observe LED transitions during rotation	Active LED changes according to relative direction	✓			Sector selection matched expected direction	
5	Rotate through full 360° motion	full 360° motion All four sector LEDs activated correctly	✓			FRONT, RIGHT, BACK, and LEFT verified as shown in fig 11 to fig 14	
6	Observe behavior near sector boundaries	LED switching remains stable without rapid flickering	✓			Hysteresis logic prevented unstable transitions	
7	Compare LED switching with heading data	Antenna changes smoothly between sectors	✓			Switching behavior consistent with algorithm	
Overall test result : Connect the UAV Location Sub-System to the ESP32-S3 via I ² C/UART, attach LEDs to GPIO pins for the four antenna sectors, upload the firmware, and rotate the prototype while monitoring heading values and LED transitions on the serial monitor. corresponding RF sectors according to the implemented truth table.			✓				

Section 5.3 Testing Full system with GNU Radio

Due to the unavailability of an RF-isolated chamber, the experiment was conducted in an open outdoor environment. Furthermore, because the required directional antennas were not available, alternative locally manufactured antennas borrowed from the College of Engineering at King Saud University were used during testing. As a result, the received signal strength was weaker than expected, which required reducing the distance between the transmitter and receiver to approximately one meter to ensure reliable measurements. The experiment was then performed for the four antenna orientations: front, right, left, and back. The measured received signal power levels were approximately -15 dB for the front direction, -22 dB for the side directions, and -28 dB for the back direction. These results demonstrate a front-to-back ratio greater than 12 dB, thereby satisfying Engineering Requirement No. 5, which states: “The system shall provide a front-to-back ratio of at least 12 dB.” Based on these results, the acceptance test table, RF power measurement table, and GNU Radio signal plots were prepared and documented, and the GNU Radio implementation used during the experiment is illustrated in the figures below.



Figure 15: Complete System Setup with Ground Control Station (GCS)

SDR Signal Processing (GNU Radio)

The GNU Radio code operates as the SDR-based testing environment for the RF part of the project. It is used to generate or receive RF signals through the SDR so that the antenna-switching behavior can be observed without needing a complete UAV flight test. In the transmitter setup, GNU Radio produces a controlled test signal that is passed through the RF switch and then routed to the selected antenna path. In the receiver setup, GNU Radio captures the received signal and displays its spectrum, power, and signal behavior, allowing the team to compare the effect of selecting different antenna sectors. This makes GNU Radio an important verification tool because it shows whether the RF subsystem responds correctly when the ESP32-S3 changes the switch selection lines. In the project, GNU Radio does not control the antenna decision directly; instead, it provides the measurement and visualization environment needed to confirm that the selected antenna path produces the expected RF output.

5.3.1 Sub-System Verification Tests

The primary function of the project is to route the transmission signal to the antenna best aligned with the receiver. To verify this, a manual geographic movement test was conducted using the SDR and GNU Radio to monitor signal strength.

Table 22: Acceptance Test Case 05 - Verification of Real-Time Antenna Sector Selection Using SDR

Test Writer: Ayman						
Test case name		RF Directionality and Dynamic Antenna Selection Verification - Engineering Requirement 5		Test ID#:		05
Description:		Prototype powered via battery. SDR transmitter generates a continuous tone using GNU Radio and feeds the RF switch input. The receiver SDR is placed at a fixed distance from the prototype to monitor Tone Power/RSSI. Serial Bluetooth Terminal is active to enter compass shift, enter GCS location, and monitor the selected antenna sector. The same distance and SDR gain settings are maintained for all receiver positions.				
Tester Information:						
Name of Tester:		Ayman, Abdullah, Abdulrahman, Mohammed		Date:		19/5/2026
Hardware Ver:		Directional Antenna Controller		Time:		10:30 pm
Setup:		Prototype powered via battery. SDR transmitting a continuous tone via GNU Radio. Receiver connected to monitor Tone Power. Serial Bluetooth Terminal active for monitoring.				
Steps	Actions	Expected Result	Pass	Fail	N/A	Comments
1	Power on the system and start the GNU Radio transmitter/receiver flowgraphs.	ESP32-S3 boots successfully, SDR tone is transmitted, GNU Radio displays a stable received tone, and Bluetooth Serial Terminal connects successfully.	✓			Baseline RF signal confirmed before movement test.
2	Enter the compass shift/calibration value if needed.	Heading shown in Bluetooth Serial Terminal is corrected and matches the prototype's real orientation.	✓			Compass offset applied before sector test.
3	Enter the location of GCS.	ESP32-S3 accepts the GCS coordinates and calculates the relative direction to the GCS.	✓			
4	Place the receiver directly in front of the active "Front" antenna sector and record the Tone Power/RSSI value in GNU Radio.	GNU Radio displays the highest or baseline Tone Power/RSSI reading for the active antenna direction.	✓			Record front-sector value as reference.
5	Move the receiver 90° to the right of the active antenna while keeping the same distance and height. antenna (outside the main lobe).	GNU Radio shows a lower Tone Power/RSSI than the front-sector baseline.	✓			Confirms signal decreases outside the main direction.
6	Move the receiver 180° behind the active antenna while keeping the	Tone Power/RSSI decreases by at least 12 dB compared with the front-sector baseline, or drops to the lowest measured level.	✓			Verifies front-to-back directionality Engineering

	same distance and height.					Requirement 5.
7	Rotate the prototype or update the GCS direction so that the selected sector changes from Front to another sector.	Bluetooth Serial Terminal shows the new selected sector, and GNU Radio shows the strongest Tone Power/RSSI when the receiver is placed in front of the newly active antenna.	✓			Confirms dynamic antenna selection, not only antenna directionality.
Overall test result: The experiment verified that the system dynamically selected the correct antenna sector and that the RF signal was strongest in the active antenna direction. Moving the receiver outside the active sector reduced the measured Tone Power/RSSI, and the rear-side measurement showed significant signal degradation compared with the front-sector baseline.			✓			

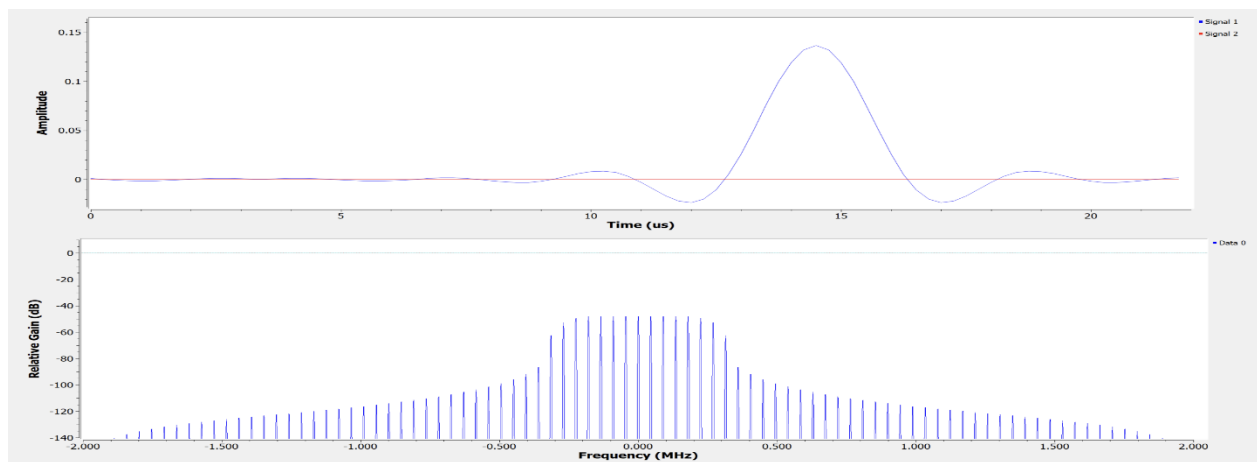


Figure 16: GNU Radio Visualization of the Transmitted RF Signal in Time and Frequency Domains

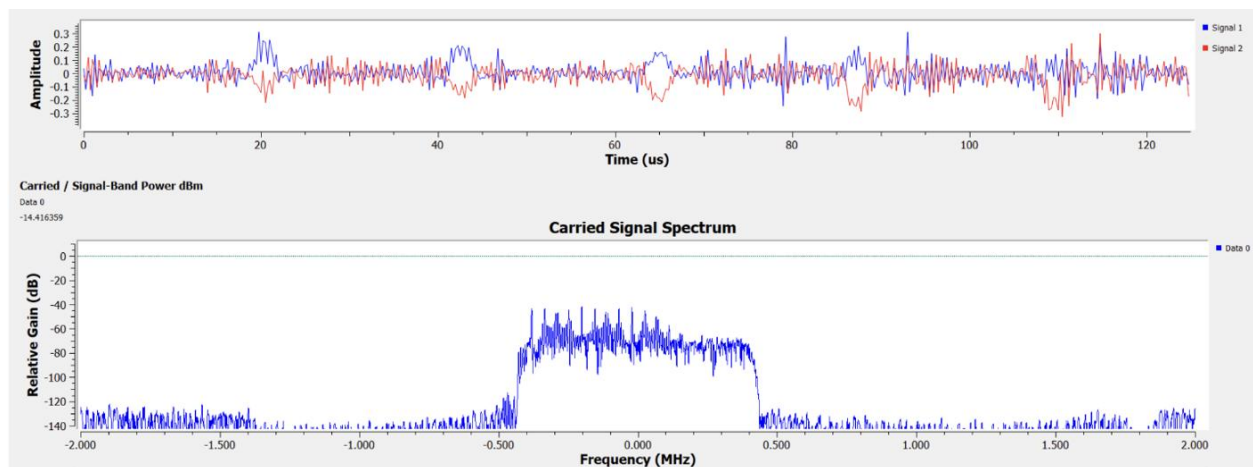


Figure 17: Received Signal Spectrum – Front Sector

Table 23: RX Tone Power for Directional Sectors

Direction	RX Tone power
Front Sector	-14.41 dBm
Right sector	-19.67 dBm
Back Sector	-27.09 dBm

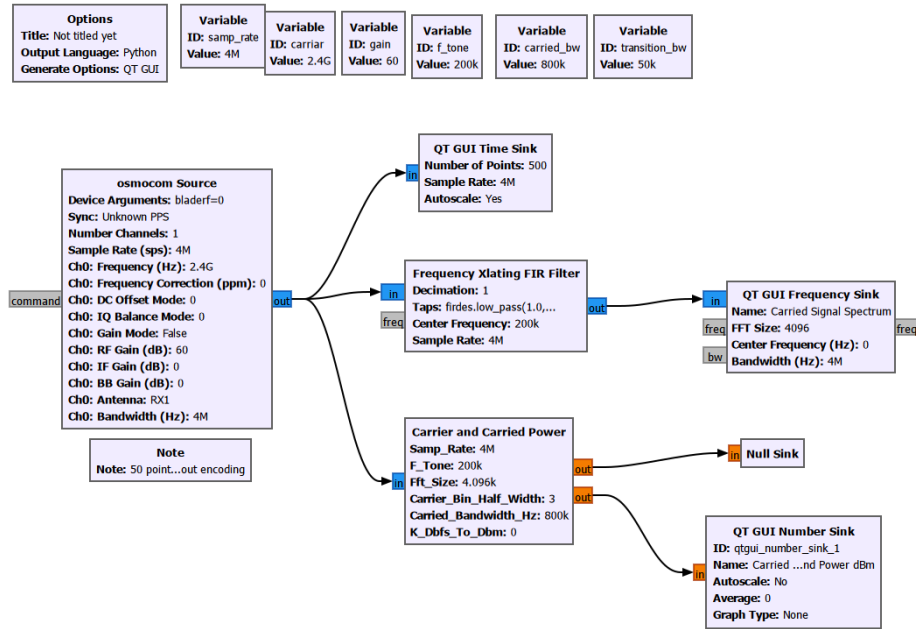


Figure 18: GNU Radio Receiver Flowgraph Implementation

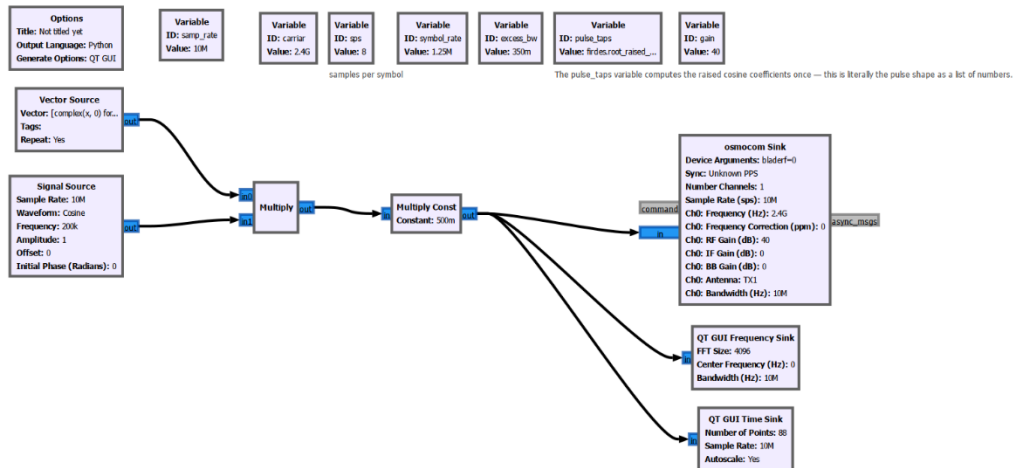


Figure 19: GNU Radio Transmit Flowgraph Implementation

References

- [1] M. M. H. Qazzaz, S. A. R. Zaidi, D. C. McLernon, A. Salama, and A. A. Al-Hameed, “Optimizing Search and Rescue UAV Connectivity in Challenging Terrain Through Multi Q-Learning,” in Proc. IEEE International Symposium on Personal, Indoor and Mobile Radio Communications (PIMRC), Valencia, Spain, 2024, pp. 1–6.
- [2] Y. Shi, R. Enami, J. Wensowitch, and J. Camp, “UABeam: UAV-Based Beamforming System—Analysis with In-Field Air-to-Ground Channels,” in Proc. IEEE International Conference on Sensing, Communication, and Networking (SECON), Hong Kong, 2018, pp. 1–9.
- [3] A. H. Kelechi, M. H. Alsharif, D. A. Oluwale, P. Achimugu, O. Ubadike, J. Nebhen, A. Aaron-Anthony, and P. Uthansakul, “The recent advancement in unmanned aerial vehicle tracking antenna: A review,” *Sensors*, vol. 21, no. 16, Art. no. 5662, 2021.
- [4] M. S. Sharawi, M. Ibrahim, S. Deif, and D. N. Aloï, “A planar printed antenna array embedded in the wing structure of a UAV for communication link enhancement,” *Progress In Electromagnetics Research*, vol. 138, pp. 697–715, 2013.
- [5] T. H. Ahmed, J. J. Tiang, A. Mahmud, C. Gwo-Chin, and D.-T. Do, “Evaluating the performance of proposed switched beam antenna systems in dynamic V2V communication networks,” *Sensors*, vol. 23, no. 15, Art. no. 6782, 2023.
- [6] P. Kant, “Embedded software of automatic sector switching antenna system for UAV data link and SOTM,” *International Journal of Computer Applications*, vol. 181, no. 7, pp. 7–12, Aug. 2018.
- [7] J. Mitola, “The software radio architecture,” *IEEE Communications Magazine*, vol. 33, no. 5,
- [8] Arduino, “Arduino Mega 2560 Hardware Documentation,” Arduino.cc. [Online]. Available: <https://docs.arduino.cc/hardware/mega-2560/>. [Accessed: 18-Dec-2025].
- [9] STMicroelectronics, “STM32F205RB Datasheet,” ST. [Online]. Available: <https://www.st.com/resource/en/datasheet/stm32f205rb.pdf>. [Accessed: 18-Dec-2025].
- [10] RUSHFPV, “RUSHFPV GNSS MINI M10,” RUSHFPV. [Online]. Available: <https://rushfpv.net/products/rushfpv-gnss-mini>. [Accessed: 18-Dec-2025].
- [11] Beitian, “Beitian BN-880 (M8Q) GNSS Module Datasheet,” Beitian. [Online]. Available: <https://images-na.ssl-images-amazon.com/images/I/81xnOf7jqyL.pdf>. [Accessed: 18-Dec-2025].
- [12] Bosch Sensortec, “BMM150 Geomagnetic Sensor Datasheet,” Bosch Sensortec. [Online]. Available: <https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bmm150-ds001.pdf>. [Accessed: 18-Dec-2025].
- [13] Honeywell, “HMC5883L 3-Axis Digital Compass IC Datasheet,” Honeywell. [Online]. Available: https://cdn-shop.adafruit.com/datasheets/HMC5883L_3-Axis_Digital_Compass_IC.pdf. [Accessed: 18-Dec-2025].
- [14] STMicroelectronics, “LPS22HB MEMS Pressure Sensor Datasheet,” ST. [Online]. Available: <https://www.st.com/resource/en/datasheet/lps22hb.pdf>. [Accessed: 18-Dec-2025].

- [15] TE Connectivity, “MS5611-01BA03 Barometric Pressure Sensor Datasheet,” TE Connectivity. [Online]. Available: https://www.te.com/commerce/DocumentDelivery/DDEController?Action=showdoc&DocId=Data+Sheet%7FMS5611-01BA03%7FB3%7Fpdf%7FEnglish%7FENG_DS_MS5611-01BA03_B3.pdf%7FCAT-BLPS0036. [Accessed: 18-Dec-2025].
- [16] Analog Devices, “HMC241AQS16E GaAs MMIC SP4T Non-Reflective RF Switch Datasheet,” Analog Devices. [Online]. Available: <https://www.analog.com/media/en/technical-documentation/data-sheets/hmc241aqs16e.pdf>. [Accessed: 18-Dec-2025].
- [17] Sevskiy GmbH, “AN220201-03C UWB Planar Log-Periodic Antenna Datasheet,” Sevskiy GmbH. [Online]. Available: <https://mm.digikey.com/Volume0/opasdata/d220001/me-dias/docus/6154/AN220201-03C.pdf>. [Accessed: 18-Dec-2025].
- [18] Taoglas, “AHP5354A.07.0100C Active Multiband GNSS Patch Antenna Datasheet,” Taoglas. [Online]. Available: <https://www.taoglas.com/datasheets/AHP5354A.07.0100C.pdf>. [Accessed: 18-Dec-2025].
- [19] SparkFun Electronics, “MMC5983MA 3-Axis Magnetometer Datasheet,” SparkFun. [Online]. Available: https://cdn.sparkfun.com/assets/a/b/7/7/2/19921-09102019_MMC5983MA_Datasheet_Rev_A-1635338.pdf. [Accessed: 18-Dec-2025].
- [20] Adafruit Industries, “BMP388, BMP390, and BMP3XX Precision Barometric Pressure Sensors,” Adafruit. [Online]. Available: <https://learn.adafruit.com/adafruit-bmp388-bmp390-bmp3xx>. [Accessed: 18-Dec-2025].
- [21] GNSS.Store, “NEO-M8N GNSS Modules,” GNSS.Store. [Online]. Available: <https://gnss.store/en/neo-m8n-gnss-modules/48-elt0035.html>. [Accessed: 18-Dec-2025].
- [22] Espressif Systems, “ESP32-S3-DevKitC-1 Hardware Documentation,” Espressif. [Online]. Available: <https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32s3/esp32-s3-devkitc-1/index.html>. [Accessed: 18-Dec-2025].
- [23] RS Components, “D481 Arduino-Compatible Breadboard Power Supply Module Datasheet,” RS-Online. [Online]. Available: <https://docs.rs-online.com/d481/0900766b81673199.pdf>. [Accessed: 17-Dec-2025].
- [24] RS Components, “A700000007484044 Universal PCB Terminal Block Datasheet,” RS-Online. [Online]. Available: <https://docs.rs-online.com/0461/A700000007484044.pdf>. [Accessed: 17-Dec-2025].
- [25] Skyworks Solutions, Inc., “SKY13370-374LF 0.5 to 6.0 GHz SPDT Switch Datasheet,” Skyworks Solutions, Inc. [Online]. Available: <https://www.skyworksinc.com/-/media/CF50141D464A4C109044EA419C90A5A1.pdf>. [Accessed: 18-Dec-2025].
- [26] Qorvo, Inc., “QPC8019Q Broadband High Frequency SPDT Switch Datasheet Rev I,” Qorvo, Inc. [Online]. Available: https://www.mouser.com/datasheet/2/412/QPC8019Q_Data_Sheet-2604536.pdf. [Accessed: 18-Dec-2025].
- [27] C. A. Balanis, *Antenna Theory: Analysis and Design*, 4th ed., Wiley, 2016. [Accessed: 9-Feb-2026].

[28] R. E. Carrel, "The Design of Log-Periodic Dipole Antennas," *IRE International Convention Record*, 1961. [Accessed: 9-Feb-2026].

Appendix A - Section A.1 Work Breakdown Structure

ID	Activity	Description	Deliverables	Duration (Days)	People	Resources	Predecessors
1	Procurement & Preparation	Plan for system hardware acquisition and SDR availability.	Ready-to-start environment.	-	-	-	-
1.1	Component Procurement	Purchasing parts (ESP32-S3, RF Switch, Antennas, Sensors) and securing the SDR device for testing.	Purchased components and SDR unit ready for lab use.	10	Ayman, Abdullah, Mohammed, Abdulrahman	Budget, Suppliers	-
1.2	Development Setup	Installing ESP-IDF for MCU and SDR software (e.g., GNU Radio, SDR#) on the testing laptop.	Configured IDE and SDR signal processing software.	2	Ayman, Abdullah	IDE, SDR Software	1.1
2	Development & Implementation	Implementation of the modules defined in Level 1 Design.	Functional software/hardware modules.	-	-	-	-
2.1	UAV Location Module Implementation	Interfacing GPS (NEO-M8N) and Compass (MMC5983MA) with MCU via I2C/UART protocols and developing the bearing calculation	Functional code reading real-time location and heading.	15	Ayman, Abdullah	ESP32, Sensors	1.2
2.2	Control Logic & Algorithm	Developing sector selection logic to drive the RF Switch.	Compiled algorithm outputting the correct antenna ID.	10	Mohammed, Abdulrahman	IDE, Math Libraries	1.2
2.3	RF Switch Integration	Constructing the RF circuit and programming the MCU to send GPIO signals to the EV1HMC241 switch.	Verified switching capability on the PCB.	20	Mohammed, Abdulrahman	RF Switch, Multimeter	2.2
3	Prototype Building	Physical assembly of the complete system.	Assembled Prototype.	-	-	-	-

3.1	Component Wiring & Assembly	Connecting the ESP32, RF Switch, and sensors using breadboards and jumper wires on the test bench.	A fully wired and functional prototype ready for testing.	14	Ayman, Abdullah, Mohammed, Abdulrahman	Breadboard, Wires	2.3
3.2	Power Supply Integration	Integrating voltage regulators to provide 5V from the battery.	Stable power distribution to all components.	4	Ayman, Abdullah, Mohammed, Abdulrahman	DC/DC Converter	3.1
4	Testing & Verification (SDR)	Verifying system performance using SDR.	Verified system with Signal Plots.	-	-	-	-
4.1	SDR Testbed Setup "receiver"	Configuring the SDR to act as a receiver/spectrum analyzer to monitor the 2.4GHz band.	Operational SDR capturing and analyzing RF signals for reception testing.	4	Ayman, Abdullah	SDR Device, Laptop	3.2
4.2	SDR Testbed Setup "Transmitter"	Configuring the SDR to act as a Transmitter (Signal Generator) to inject a test signal into the RF Switch.	Operational SDR generating a stable RF signal for transmission testing.	4	Mohammed, Abdulrahman	SDR Device, Laptop	3.2
4.3	Beam Switching Verification	Using SDR to measure RSSI changes and SNR when the MCU switches antenna sectors.	Test report with RSSI plots confirming beam steering.	8	Ayman, Abdullah, Mohammed, Abdulrahman	SDR, GNU Radio	4.1, 4.2
4.4	System Response Test	Verifying the time delay between coordinate change and signal detection on the SDR.	Validation that response time is <100ms.	6	Ayman, Abdullah, Mohammed, Abdulrahman	SDR, Logic Analyzer	4.3

Section A.2 Gantt Chart

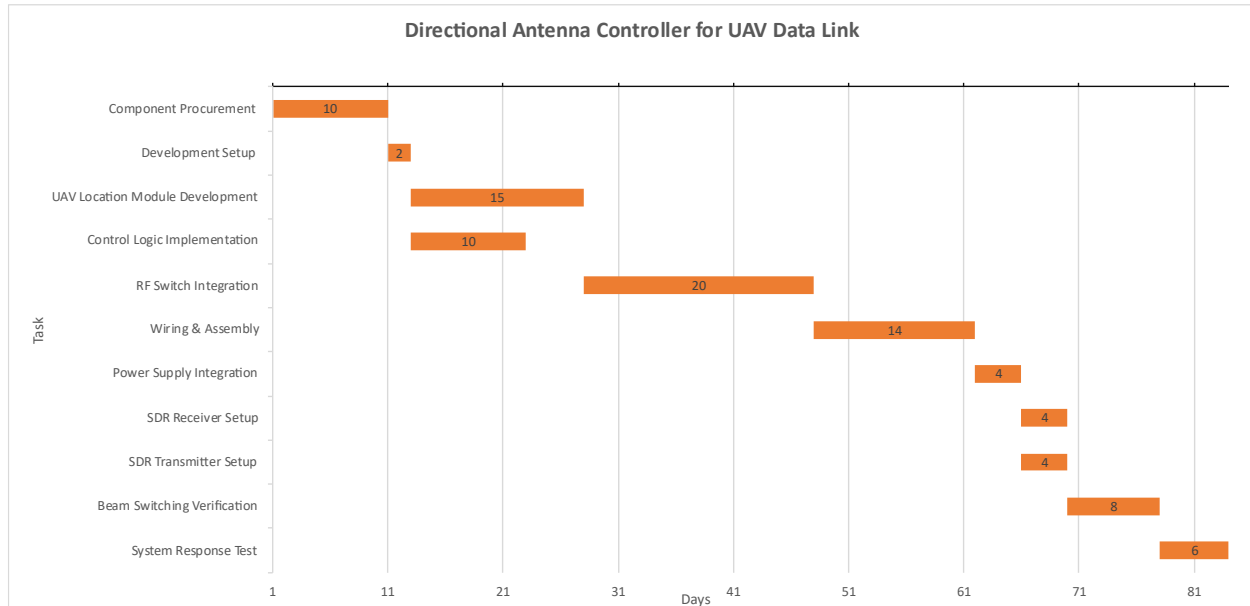


Figure 20: Gantt Chart

Section A.3 Cost Estimation

Item no.	Name	Quantity	Cost	Total cost
1	ESP32-S3 DevKitC-1	1	56	56
2	NEO-M8N GPS Module	1	300	300
3	BMP390 Barometric Sensor	1	41	41
4	MMC5983MA Magnetometer	1	69	69
5	External GPS Antenna	1	150	150
6	JST-SH 4-pin Cable (Qwiic)	1	13	13
7	JST-SH 6-pin Cable	1	17	17
8	U.FL to SMA Pigtail 10cm	1	18.75	18.75
9	AN220201-03C Antenna	4	225	900
10	RF Switch SP4T	1	495	495
11	Dupont Jumper Wires	1	20	20
12	SMA Male-to-Male 25cm	5	18.6	93

Total cost: 2172.75

Appendix B Supplementary information

Item no.	Component type	Component Name	Datasheet Link
1	MCU	ESP32-S3 DevKitC-1	Datasheet
2	GPS	NEO-M8N GPS Module	Datasheet
3	Altitude	BMP390 Barometric Sensor	Datasheet
4	Compass	MMC5983MA Magnetometer	Datasheet
5	External GPS Antenna	AHP5354A.07.0100C	Datasheet
6	Antenna	AN220201-03C Antenna	Datasheet
7	RF switch	RF Switch SP4T	Datasheet

Appendix C – Concept Evaluation

MCU (AHP):

		ESP32-S3	STM32	Arduino Mega
Real-time control	0.45	0.37	0.47	0.16
Size	0.25	0.50	0.38	0.12
Performance	0.20	0.50	0.39	0.11
Cost	0.10	0.30	0.25	0.45
Score		0.422	0.410	0.168

RF Switch (SWA):

Option	Strengths	Weaknesses	Best Choice
EV1HMC241AQS16	Extremely low insertion loss, high isolation	Higher cost and requires RF-grade PCB layout/biasing to achieve low loss and high isolation.	✓
SKY13370-370LF	Good balance between performance and cost	Standard isolation compared to high-end options	
QPC8019Q	Optimized for high-power applications	Requires complex cascading to fit 4 antennas	

Antenna (SWA):

Option	Strengths	Weaknesses	Best Choice
Log-Periodic (AN220201)	Aerodynamic (flat), high gain, wide bandwidth	Larger footprint than a patch; mounting/clearance and coax routing can affect the radiation pattern.	✓
Yagi-Uda Antenna	Excellent range and high gain	High aerodynamic drag, physically fragile structure	
Ceramic Patch Antenna	Compact and very low-profile	Narrow frequency bandwidth, lower gain	

GPS (SWA):

Option	Strengths	Weaknesses	Best Choice
NEO-M8N	Fast acquisition, stable accuracy, simple integration	Can be impacted by multipath/poor sky view, increasing bearing latency (especially during aggressive maneuvers)	✓
RUSHFPV GNSS M10	Compact, high update rates (10 Hz), low power	Less documented, higher cost	
Beitian BN-880	Combined GPS + Compass module	Bulkier, less suited for aerodynamic efficiency	

Digital Compass (SWA):

Option	Strengths	Weaknesses	Best Choice
MMC5983MA	Ultra-low noise, high resolution, superior stability	Sensitive to magnetic interference from UAV motors/power wiring; requires calibration and careful placement.	✓
BMM150	Compact, low power, I ² C compatibility	Lower precision compared to MMC5983MA	
HMC5883L	Popular and widely used sensor	Discontinued, lower resolution and stability	

Altitude Module (SWA):

Option	Strengths	Weaknesses	Best Choice
BMP390	High precision ± 0.03 hPa, low drift, low power	Affected by air-flow/prop wash and weather; needs shielding/filtering.	✓
LPS22HB	Low cost, simple integration	Lower accuracy ± 0.1 hPa	
MS5611	High precision ± 0.01 hPa	Higher power consumption, more expensive	

Appendix D Program code

D.1 ESP32 Code :

ESP32-S3 Antenna Controller for UAV Code

```
#include <Wire.h>
#include <Adafruit_Sensor.h>
#include "Adafruit_BMP3XX.h"
#include <HardwareSerial.h>
#include <TinyGPSPlus.h>
#include <SparkFun_MMC5983MA_Arduino_Library.h>
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>
#include <Preferences.h>

// — Pin Definitions —
#define BMP_SDA 8
#define BMP_SCL 9
#define MMC_SDA 4
#define MMC_SCL 5
#define GPS_RX 16
#define GPS_TX 17
#define SWITCH_A 38
#define SWITCH_B 39
#define SEALEVELPRESSURE_HPA 1013.25

// — Timing Definitions —
// Sensor data is refreshed every 100 ms.
// Antenna switching is checked every 100 ms.
// Bluetooth/Serial report is sent every 5000 ms.
#define SENSOR_PERIOD_MS 100
#define SWITCH_PERIOD_MS 100
#define REPORT_PERIOD_MS 5000

// — BLE UUIDs —
#define SERVICE_UUID "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
#define CHARACTERISTIC_UUID_RX "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"
#define CHARACTERISTIC_UUID_TX "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"

// — Sensor Objects —
Adafruit_BMP3XX bmp;
SFE_MMC5983MA mag;
TinyGPSPlus gps;
HardwareSerial gpsSerial(1);
TwoWire I2C_0 = TwoWire(0);
TwoWire I2C_1 = TwoWire(1);

// — BLE Objects —
BLEServer* pServer = NULL;
BLECharacteristic* pTxCharacteristic = NULL;
bool deviceConnected = false;
```

```

// — Setup State Machine —————
enum SetupState { WAIT_CONNECT, CALIBRATING, WAIT_GCS, RUNNING };
SetupState setupState = WAIT_CONNECT;

// — Persistent Storage —————
Preferences prefs;

// — Antenna Offset —————
float BOARD_OFFSET = 0.0;

// — Antenna Hysteresis —————
#define HYSTERESIS_DEG 5.0f
String currentAntenna = "FRONT";

// — Shared Data Struct —————
struct SensorData {
    float heading;
    float boardLat;
    float boardLon;
    float temperature;
    float altitude;
    float bearingToGCS;
    float relativeAngle;
    bool  gpsValid;
};

// — GCS Location —————
float GCS_LAT = 0.0;
float GCS_LON = 0.0;
bool  gcsSet  = false;

// — Timing Variables —————
unsigned long lastSendTime    = 0;    // Used for calibration heading messages
unsigned long lastSensorTime = 0;    // Used for sensor reading
unsigned long lastSwitchTime = 0;    // Used for fast antenna switching
unsigned long lastReportTime = 0;    // Used for slow Bluetooth/Serial report

SensorData data;

// =====
// FUNCTION 1 – COMMAND HANDLER
// Input : Bluetooth link, messages containing GCS location update
// Output: Command output, location of GCS
// =====

void bleSend(String msg) {
    if (deviceConnected && pTxCharacteristic) {
        pTxCharacteristic->setValue(msg.c_str());
        pTxCharacteristic->notify();
    }
}

class MyServerCallbacks : public BLEServerCallbacks {
    void onConnect(BLEServer* pServer) {

```



```

deviceConnected = true;
setupState = CALIBRATING;

Serial.println("[BLE] Phone connected - starting calibration.");

delay(300);

bleSend("=== STEP 1: COMPASS CALIBRATION ===");
bleSend("Restored offset from last session: " + String(BOARD_OFFSET, 1) + " deg");
bleSend("Slowly rotate the board 360 degrees.");
bleSend("Send a number like -3 or +5 to adjust offset.");
bleSend("Reply OK when heading looks correct.");
}

void onDisconnect(BLEServer* pServer) {
    deviceConnected = false;
    setupState = WAIT_CONNECT;

    Serial.println("[BLE] Disconnected - offset preserved. Restarting advertising...");

    pServer->startAdvertising();
}

};

bool parseGCS(String incoming, float& lat, float& lon) {
    int commaIndex = incoming.indexOf(',');

    if (commaIndex == -1) return false;

    lat = incoming.substring(0, commaIndex).toFloat();
    lon = incoming.substring(commaIndex + 1).toFloat();

    if (lat == 0.0 && lon == 0.0) return false;
    if (lat < -90.0 || lat > 90.0) return false;
    if (lon < -180.0 || lon > 180.0) return false;

    return true;
}

class MyCallbacks : public BLECharacteristicCallbacks {
    void onWrite(BLECharacteristic* pCharacteristic) {
        String incoming = pCharacteristic->getValue().c_str();
        incoming.trim();

        // — Step 1: Calibration —————
        if (setupState == CALIBRATING) {
            if (incoming.equalsIgnoreCase("OK")) {
                setupState = WAIT_GCS;

                bleSend("Calibration confirmed! Final offset: " + String(BOARD_OFFSET, 1) + " deg");

                delay(200);
            }
        }
    }
};

```

```

        bleSend("=== STEP 2: GCS LOCATION ===");
        bleSend("Send ground station coordinates as: LAT,LON");
        bleSend("Example: 24.688000,46.722000");

        if (gcsSet) {
            bleSend("Previous GCS still loaded: " + String(GCS_LAT, 6) + "," +
String(GCS_LON, 6));
            bleSend("Send KEEP to reuse it, or send new LAT,LON.");
        }
    }
    else if (incoming.equalsIgnoreCase("KEEP")) {
        bleSend("KEEP is only valid after calibration is confirmed. Send OK first.");
    }
    else {
        bool isZero = (incoming == "0" || incoming == "0.0" ||
incoming == "+0" || incoming == "-0");

        float adjustment = incoming.toFloat();

        if (adjustment != 0.0 || isZero) {
            BOARD_OFFSET = fmod(BOARD_OFFSET + adjustment + 360.0, 360.0);

            prefs.putFloat("offset", BOARD_OFFSET);

            bleSend("Offset adjusted by " + String(adjustment, 1) + " deg");
            bleSend("New offset: " + String(BOARD_OFFSET, 1) + " deg");
            bleSend("Keep adjusting or reply OK to confirm.");
        }
        else {
            bleSend("Not recognised. Send a number like -3, or OK to confirm.");
        }
    }
}

return;
}

// — Step 2: GCS Coordinates —————
if (setupState == WAIT_GCS) {
    if (incoming.equalsIgnoreCase("KEEP") && gcsSet) {
        setupState = RUNNING;

        bleSend("Reusing GCS: " + String(GCS_LAT, 6) + ", " + String(GCS_LON, 6));
        bleSend("=== SETUP COMPLETE — Antenna tracking started ===");

        return;
    }
}

float lat, lon;

if (parseGCS(incoming, lat, lon)) {
    GCS_LAT = lat;
    GCS_LON = lon;
    gcsSet = true;
    setupState = RUNNING;
}

```

```

        String confirm = "GCS SET: " + String(GCS_LAT, 6) + ", " + String(GCS_LON, 6);

        Serial.println(confirm);
        bleSend(confirm);
        bleSend("=== SETUP COMPLETE - Antenna tracking started ===");
    }
    else {
        bleSend("ERROR: Invalid coordinates. Format must be LAT,LON (e.g.
24.688000,46.722000)");
    }

    return;
}

// — Running: Allow GCS Update —————
if (setupState == RUNNING) {
    if (incoming == "-1") {
        setupState = WAIT_GCS;

        bleSend("=== GCS UPDATE ===");
        bleSend("Send new coordinates as: LAT,LON");
        bleSend("Example: 24.688000,46.722000");
        bleSend("Or send KEEP to keep current: " + String(GCS_LAT, 6) + ", " +
String(GCS_LON, 6));

        return;
    }

    float lat, lon;

    if (parseGCS(incoming, lat, lon)) {
        GCS_LAT = lat;
        GCS_LON = lon;

        String confirm = "GCS UPDATED: " + String(GCS_LAT, 6) + ", " + String(GCS_LON,
6);

        Serial.println(confirm);
        bleSend(confirm);
    }
    else {
        bleSend("ERROR: Format must be LAT,LON (e.g. 24.688000,46.722000)");
    }
}
};

void setupBluetooth() {
    BLEDevice::init("ESP32-S3_BLE");

    pServer = BLEDevice::createServer();
    pServer->setCallbacks(new MyServerCallbacks());

    BLEService* pService = pServer->createService(SERVICE_UUID);

```

```

pTxCharacteristic = pService->createCharacteristic(
    CHARACTERISTIC_UUID_TX,
    BLECharacteristic::PROPERTY_NOTIFY
);

pTxCharacteristic->addDescriptor(new BLE2902());

BLECharacteristic* pRxCharacteristic = pService->createCharacteristic(
    CHARACTERISTIC_UUID_RX,
    BLECharacteristic::PROPERTY_WRITE
);

pRxCharacteristic->setCallbacks(new MyCallbacks());

pService->start();
pServer->getAdvertising()->start();

Serial.println("[BLE] Started - Device: ESP32-S3_BLE");
}

// =====
// FUNCTION 2 - GEO LOCATION STATE MEASUREMENT
// Input : GPS data, heading data, altitude data
// Output: UAV_state: lat, lon, altitude, heading
// =====

#define EMA_ALPHA 0.50f

float emaX      = 0.0f;
float emaY      = 0.0f;
bool  emaSeeded = false;

SensorData readSensors() {
    SensorData d = {};

    if (bmp.performReading()) {
        d.temperature = bmp.temperature;
        d.altitude     = bmp.readAltitude(SEALEVELPRESSURE_HPA);
    }

    uint32_t rawX = 0, rawY = 0, rawZ = 0;

    mag.getMeasurementXYZ(&rawX, &rawY, &rawZ);

    float magX = ((float)rawX - 131072.0f) / 16384.0f;
    float magY = ((float)rawY - 131072.0f) / 16384.0f;

    if (!emaSeeded) {
        emaX      = magX;
        emaY      = magY;
        emaSeeded = true;
    }
    else {
        emaX = EMA_ALPHA * magX + (1.0f - EMA_ALPHA) * emaX;

```

```

    emaY = EMA_ALPHA * magY + (1.0f - EMA_ALPHA) * emaY;
}

float heading = degrees(atan2(emaY, emaX));

if (heading < 0) {
    heading += 360.0f;
}

d.heading = fmod(heading - BOARD_OFFSET + 360.0f, 360.0f);

d.gpsValid = gps.location.isValid();

if (d.gpsValid) {
    d.boardLat = gps.location.lat();
    d.boardLon = gps.location.lng();
}

return d;
}

// =====
// FUNCTION 3 - COMPUTATION ALGORITHM
// Input : UAV_state + GCS coordinates
// Output: Relative deg
// =====

float computeRelativeDeg(SensorData& d) {
    if (!gcsSet) {
        d.relativeAngle = -1.0f;
        return d.relativeAngle;
    }

    if (!d.gpsValid) {
        d.relativeAngle = -2.0f;
        return d.relativeAngle;
    }

    float dLon = radians(GCS_LON - d.boardLon);
    float lat1 = radians(d.boardLat);
    float lat2 = radians(GCS_LAT);

    float x = sin(dLon) * cos(lat2);

    float y = cos(lat1) * sin(lat2) -
        sin(lat1) * cos(lat2) * cos(dLon);

    d.bearingToGCS = degrees(atan2(x, y));

    if (d.bearingToGCS < 0) {
        d.bearingToGCS += 360.0f;
    }

    d.relativeAngle = fmod(d.bearingToGCS - d.heading + 360.0f, 360.0f);
}

```

```

    return d.relativeAngle;
}

// =====
// FUNCTION 4 – ANTENNA SELECTION SUBSYSTEM
// Input : Relative deg
// Output: Selection line 1 and Selection line 2
// =====

void applyAntenna(const String& sector) {
    if (sector == "FRONT") {
        digitalWrite(SWITCH_A, LOW);
        digitalWrite(SWITCH_B, LOW);
    }
    else if (sector == "RIGHT") {
        digitalWrite(SWITCH_A, HIGH);
        digitalWrite(SWITCH_B, LOW);
    }
    else if (sector == "BACK") {
        digitalWrite(SWITCH_A, LOW);
        digitalWrite(SWITCH_B, HIGH);
    }
    else {
        digitalWrite(SWITCH_A, HIGH);
        digitalWrite(SWITCH_B, HIGH);
    }
}

String rawSector(float relativeDeg) {
    if (relativeDeg >= 315.0f || relativeDeg < 45.0f) {
        return "FRONT";
    }

    if (relativeDeg < 135.0f) {
        return "RIGHT";
    }

    if (relativeDeg < 225.0f) {
        return "BACK";
    }

    return "LEFT";
}

float sectorCentre(const String& sector) {
    if (sector == "FRONT") return 0.0f;
    if (sector == "RIGHT") return 90.0f;
    if (sector == "BACK") return 180.0f;

    return 270.0f;
}

String antennaSelectionSubsystem(float relativeDeg) {

```

```

    if (relativeDeg == -1.0f) {
        return "NO_GCS";
    }

    if (relativeDeg == -2.0f) {
        return "NO_GPS";
    }

    String candidate = rawSector(relativeDeg);

    if (candidate != currentAntenna) {
        float centre = sectorCentre(candidate);

        float fromCentre = fmod(fabs(relativeDeg - centre) + 360.0f, 360.0f);

        if (fromCentre > 180.0f) {
            fromCentre = 360.0f - fromCentre;
        }

        float penetration = 45.0f - fromCentre;

        if (penetration >= HYSTERESIS_DEG) {
            currentAntenna = candidate;
            applyAntenna(currentAntenna);
        }
    }

    return currentAntenna;
}

// =====
//  OUTPUT
//  Input : UAV_state + selected antenna
//  Output: Command output over Bluetooth / Serial
//  =====

void sendOutput(SensorData& d, const String& antenna) {
    String msg;

    if (antenna == "NO_GCS") {
        msg = "Waiting for GCS coordinates...";
    }
    else if (antenna == "NO_GPS") {
        msg = "No GPS fix yet...";
    }
    else {
        msg += "Antenna : " + antenna + "\n";
        msg += "Heading : " + String(d.heading, 1) + " deg\n";
        msg += "Bearing : " + String(d.bearingToGCS, 1) + " deg\n";
        msg += "Rel Ang : " + String(d.relativeAngle, 1) + " deg\n";
        msg += "Lat      : " + String(d.boardLat, 6) + "\n";
        msg += "Lon      : " + String(d.boardLon, 6) + "\n";
        msg += "Temp     : " + String(d.temperature, 2) + " C\n";
        msg += "Alt      : " + String(d.altitude, 2) + " m\n";
    }
}

```

```

    Serial.println(msg);
    bleSend(msg);
}

// =====
// MAIN LOOP FUNCTION 1 – READING BLUETOOTH
// Module : Command Handler
// Input  : Bluetooth link / command input
// Output : Command output + location of GCS
// =====

void CommandHandler() {
    if (setupState == CALIBRATING && millis() - lastSendTime >= 1000) {
        lastSendTime = millis();

        bleSend("Heading: " + String(data.heading, 1) +
                " deg | Send a number to adjust or OK to confirm");
    }
}

// =====
// MAIN LOOP FUNCTION 2 – READING DATA SENSOR
// Module : Geo location state measurement
// Input  : GPS data, heading data, altitude data
// Output : UAV_state: lat, lon, altitude, heading
// =====

void readingDataSensor() {
    while (gpsSerial.available()) {
        gps.encode(gpsSerial.read());
    }

    if (millis() - lastSensorTime >= SENSOR_PERIOD_MS) {
        lastSensorTime = millis();

        data = readSensors();
    }
}

// =====
// MAIN LOOP FUNCTION 3 – COMPUTATION
// Module : Computation algorithm
// Input  : UAV_state + location of GCS
// Output : Relative deg
// =====

void computation() {
    if (setupState == RUNNING) {
        computeRelativeDeg(data);
    }
}

```



```

// =====
// MAIN LOOP FUNCTION 4 - OUTPUT
// Module : Antenna selection subsystem + output
// Input  : Relative deg
// Output : Selection line 1 & 2 + command output
// =====

void output() {
  if (setupState != RUNNING) {
    return;
  }

  // Fast path:
  // Command antenna-sector switching every 100 ms.
  // This satisfies the ≤ 500 ms sensing-to-switching requirement.
  if (millis() - lastSwitchTime >= SWITCH_PERIOD_MS) {
    lastSwitchTime = millis();

    antennaSelectionSubsystem(data.relativeAngle);
  }

  // Slow path:
  // Send Bluetooth/Serial status report every 5 seconds.
  // This does not delay antenna switching.
  if (millis() - lastReportTime >= REPORT_PERIOD_MS) {
    lastReportTime = millis();

    String selectedAntenna = antennaSelectionSubsystem(data.relativeAngle);

    sendOutput(data, selectedAntenna);
  }
}

// =====
// SETUP
// =====

void setup() {
  Serial.begin(115200);

  while (!Serial) {
    delay(10);
  }

  prefs.begin("tracker", false);
  BOARD_OFFSET = prefs.getFloat("offset", 0.0);

  Serial.println("[NVS] Loaded compass offset: " + String(BOARD_OFFSET, 1) + " deg");

  I2C_0.begin(BMP_SDA, BMP_SCL);

  if (!bmp.begin_I2C(0x77, &I2C_0)) {
    Serial.println("[BMP390] Not found! Check wiring on SDA=8, SCL=9.");

    while (1) {

```

```

        delay(10);
    }
}

bmp.setTemperatureOversampling(BMP3_OVERSAMPLING_8X);
bmp.setPressureOversampling(BMP3_OVERSAMPLING_4X);
bmp.setIIRFilterCoeff(BMP3_IIR_FILTER_COEFF_3);
bmp.setOutputDataRate(BMP3_ODR_50_HZ);

Serial.println("[BMP390] OK");

I2C_1.begin(MMC_SDA, MMC_SCL);

if (!mag.begin(I2C_1)) {
    Serial.println("[MMC5983MA] Not found! Check wiring on SDA=4, SCL=5.");

    while (1) {
        delay(10);
    }
}

mag.softReset();

Serial.println("[MMC5983MA] OK");

gpsSerial.begin(9600, SERIAL_8N1, GPS_RX, GPS_TX);

Serial.println("[GPS] UART started on RX=16, TX=17");

pinMode(SWITCH_A, OUTPUT);
pinMode(SWITCH_B, OUTPUT);

digitalWrite(SWITCH_A, LOW);
digitalWrite(SWITCH_B, LOW);

setupBluetooth();

Serial.println("=== Ready - waiting for BLE connection ===");
}

// =====
//  MAIN LOOP
// =====

void loop() {
    CommandHandler();
    readingDataSensor();
    computation();
    output();
}

```